Introduction to Web Engineering

Week-1

Introduction Session

- **Introduce yourself:**
  - Name, majors, expertise, experiences, keywords related to web engineering.

Software Project Success Rate:



⇨ Data on 280,000 projects completed in 2000 - *Standish Group Data*.

http://www.softwaremag.com/archive/2001feb/CollaborativeMgt.html

Definition of Software Engineering:

- *Software Engineering is defined as the application of science and mathematics by which the capabilities of computer equipment are made useful to human via computer programs, procedures, and associated documentation.*

A Survey by the Cutter Consortium:

- The top problem areas of large-scale web application projects are:
  - Failure to meet business needs (84%)
  - Project schedule delays (79%)
  - Budget overrun (63%)
  - Lack of functionalities (53%)
  - Poor quality of deliverables (52%)

Definition of Web Engineering:

- *Web Engineering is the application of systematic and quantifiable approaches (concepts, methods, techniques, tools) to cost-effective requirements analysis, design, implementation, testing, operation, and maintenance of high-quality web applications.*

Chapter-1: Web-Based Systems

- In the early days of the web, we built web based systems using informality, urgency, intuition, and art.
  - ***Informality*** leads to an easy-work environment in which you can do your own thing.
  - ***Urgency*** leads to rapid action and decision making.
  - ***Intuition*** is an intangible quality that enables you to "feel" your way through complex situations.
  - ***Art*** leads to aesthetic form and function to something that pleases those who encounter it.
- This approach can and often does lead to some problems.

The Web

- An indispensable technology in virtually every aspect of modern living.
- A transformative technology that
  - changes the way we do things and
  - changes the way we acquire and disseminate information.
- It is an evolving technology.
- It has high impact on everything in the modern world.

Web Apps

- The term *Web application* encompasses:
  - everything from a simple Web page that might help a consumer compute an automobile lease payment to a comprehensive website that provides complete travel services for business people and vacationers.
  - Included within this category are complete websites, specialized functionality within websites, and information-processing applications that reside on the Internet or on an Intranet or Extranet.

Web Application Attributes

- Network intensiveness
- Concurrency
- Unpredictable load
- Performance
- Aesthetics
- Availability
- Content sensitive
- Continuous evolution
- Immediacy
- Security

- Data driven

Web Application Types

- Informational
- Document-centric
- Customizable
- Interactive
- Workflow-based
- Social-oriented
- Ubiquitous
- Transactional
- Service-oriented
- Portal-oriented
- Collaborative
- Semantic

Why Web Engineering?

- As web applications become larger and more complex:
  - Informality remains but some degree of requirements gathering and planning are necessary.
  - Urgency remains but it must be tempered by a recognition that decisions may have broad consequences.
  - Intuition remains but it must be augmented by proven management and technical patterns.
  - Art remains but it must be complemented with solid design.

- We must adapt the old-school approach to the realities of a web 2.0 world.

Chapter 2: *Web Engineering*

- We define web engineering in this way:
  - *An agile, yet disciplined framework for building industry-quality Web Applications.*

- We must understand the meaning of:
  - Agile
  - Disciplined framework
  - Industry quality

Why Agility?

- Business strategies and rules change rapidly.
- Management demands near-instantaneous responsiveness (even when such demands are completely unreasonable).
- Stakeholders often don’t understand the consequences of the web application and keep changing their mind even as they demand rapid delivery.
- An agile approach helps to cope with this fluidity and uncertainty.

What is an Agile Process?

- Agile web engineering process combines a philosophy and a set of development guidelines. The philosophy encourages:
  - Customer's satisfaction
  - early incremental delivery of the web applications
  - small, highly motivated project teams
  - informal methods
  - minimal work products
  - overall development simplicity.

What is an Agile Process?

- An agile process stresses delivery over analysis and design (although these activities are not discouraged), active and continuous communication between developers and customers.

- http://agilemanifesto.org/

Underlying Agility Principles-1

- Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.
- Always welcome changing requirements, even late in development. Agile processes harness continuous change for the customer's competitive advantage.
- Deliver working software increments frequently, from as often as every few days to every few months, with a preference to the shorter timescales.
- Business people and developers must work together daily throughout the project.
- Build projects around motivated people. Give them the environment and support they need and trust them to get the job done. The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.

Underlying Agility Principles-2

- Working software is the primary measure of progress.
- Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
- Continuous attention to technical excellence and good design enhances agility.
- Simplicity, defined as the practice of maximizing the amount of work not undertaken, is a fundamental principle of effective software development.
- The best architectures, requirements, and designs emerge from self- organizing teams.
- At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

What is a WebE Framework?

- A framework is a set of activities that will always be performed for every web engineering project, though the nature of the activities might vary to suit the project.
- Each framework activity is composed of a set of actions.
- Actions encompasses
  - work tasks
  - work products
  - quality assurance points
  - project milestones
- A framework also has a set of “umbrella activities”

WebE Framework Activities

- **Communication:** It involves heavy interaction and collaboration with the customer and other stakeholders and encompasses requirements gathering and other related activities.
- **Planning:** Establishes an incremental plan for the web engineering work.
- **Modeling:** Encompasses the creation of models that assist the developer and the customer to better understand web application's requirements and the design.
- **Construction:** Combines both the generation of HTML, XML, Java, JavaScript and similar code with testing that is required to uncover errors in the code.
- **Deployment:** Delivers a web application increment to the customer who evaluates it and provides feedback based on the evaluation.

Adapting the Framework

- Adapt to the problem, to the project, to the team, and to the organizational culture and continue to adapt throughout the project as circumstances change!

- Adaptation leads to:
  - Overall flow of activities, actions, and tasks and the interdependencies among them.
  - Degree to which work tasks are defined within each framework activity.
  - Degree to which work products are identified and required.
  - Manner in which quality assurance activities are applied.
  - Manner in which project tracking and control activities are applied.
  - Overall degree of detail and rigor with which the process is described.
  - Degree to which customers and other stakeholders are involved with the project.
  - Level of autonomy given to the software project team.
  - Degree to which team organization and roles are prescribed.

The Influence of Software Engineering



- **Quality:** Foster a continuous process improvement culture.
- **Process:** The glue that holds the technology layers together.
- **Methods:** Provide the technical how-to's.
- **Tools:** Support for the process and the methods.

Web Engineering Methods

- Communication methods
- Requirements analysis methods
- Design methods
- Construction methods
- Testing methods

What about Tools and Technology?

... tools and technology are very important, but they'll work well only if they're used within the context of an agile framework for web engineering and in conjunction with proven methods for understanding the problem, designing a solution, and testing it thoroughly.

WebE Best Practices

- Take the time to understand business needs and product objectives, even if the details of the web applications are vague.
- Describe how users will interact with the web applications using a scenario-based approach.
- Always develop a project plan, even if it's very brief.
- Spend some time for modeling what it is that you're going to build.
- Review the models for consistency and quality.
- Use tools and technology that enable you to construct the system with as many reusable components as possible.
- Don't reinvent when you can reuse.
- Don't rely on early users to debug the web applications, design and use comprehensive tests before releasing the system.

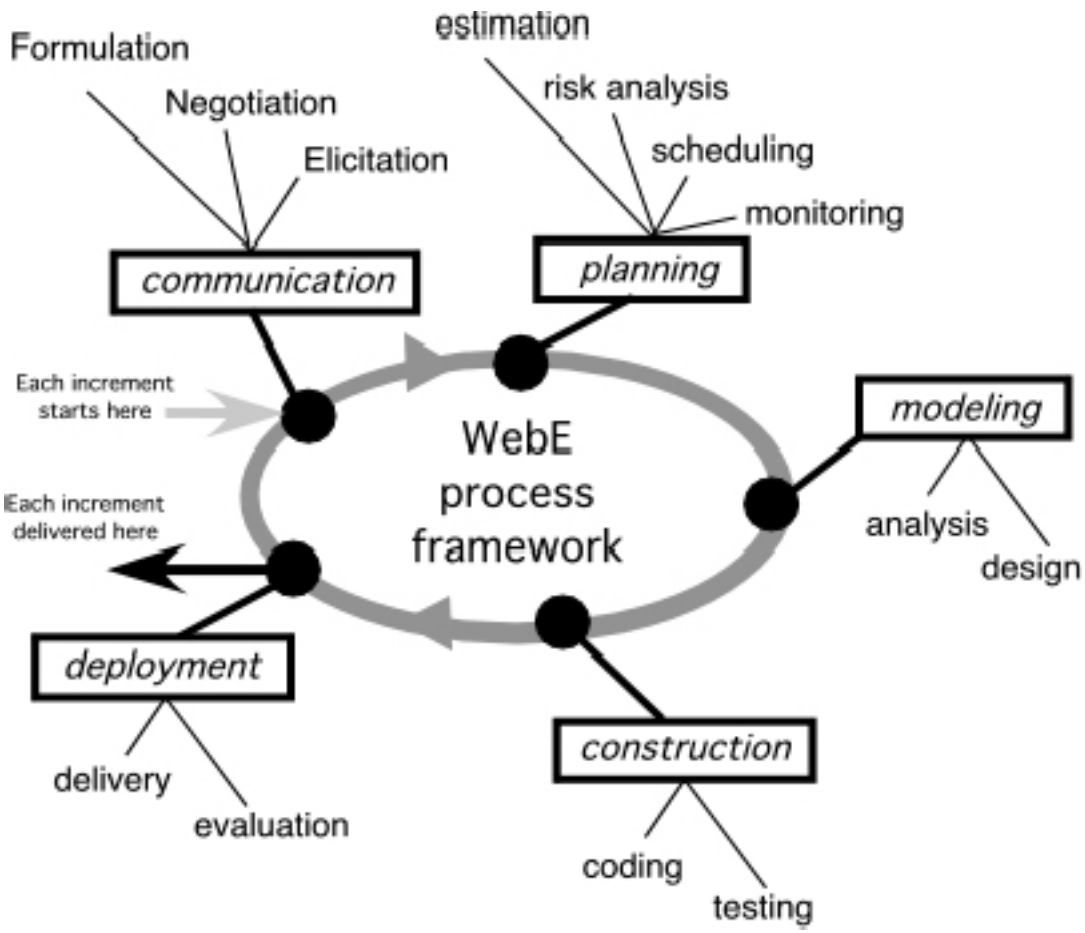
Web Engineering Process

Week-2

Web Engineering Process

- The process must be agile and adaptable, but it must also be *incremental.*
- Why incremental?
  - Requirements evolve over time
  - Changes will occur frequently and always at inconvenient times
  - Time lines are short
- Incremental delivery allows you to manage this change!

WebE Process Activities & Actions



Conducting Framework Activities-I

- The first iteration
  - define business context
  - establish overall requirements
  - create a set of usage scenarios
  - negotiate conflicting needs among stakeholders
  - from this information derive the set of web application increments that is to be delivered.
- Develop a broad outline of all components, recognizing that it will change.

Conducting Framework Activities-II

- The second iteration:
  - You've learned that the first increment is an informational Web Application and it must be delivered in one week!
  - You meet with stakeholders and later review your notes:
    - Logo and graphics—need aesthetic design.
    - One- or two-paragraph introduction.
    - CPI mission statement (file exists)
    - A word to visitors (someone will write this tomorrow)
    - Basic navigation bar will look like …
      - About the company
      - Our offerings
      - Home security products (hierarchical at next level)
      - Monitoring services (a list)
      - Our Technology (the new sensor)
      - Contact us
    - Other issues:
    - Informational content will change over time.
      - This "home page" will be the navigation starting point for content and functions required for subsequent increments.

Conducting Framework Activities-III

- The second iteration:
  - You spend a few minutes developing a plan
    - **Day-1**: Create a prototype layout (a model) of the web application.
      - Collect and review all existing CPI content and graphics.
      - Get stakeholder feedback on prototype, if possible.
    - **Day 2:** Using the prototype as a guide, begin construction of the increment.
      - Build navigation bar, layout content areas.
      - Integrate graphics, links etc.
      - Test all links for validity.
      - Review all content for completeness and correctness.
    - **Day 3:** FTP all files to an existing domain.
      - Perform navigation tests.
      - **Deployment:** Inform selected stakeholders that the increment is available.
    - **Day 4:** Poll stakeholders for feedback.
      - Make modifications based on stakeholder's feedback.

Conducting Framework Activities-IV

- The next iteration
  - You've deployed the informational web application.
- The communication activity during this second iteration will identify the requirements (including content and functionality)
  - assume that the second increment delivers the capability to select and download product specifications and related information
- The process flow is restarted at the beginning, performing the communication activity for this increment.
- The tasks you select to populate each framework activity for the increment may differ from the tasks performed for the preceding increment, but the overall process flow remains the same.

Revisiting the Framework Activities

- Web App (32 - 42) presents a breakdown of the generic actions and tasks for each of the five framework activities.
- Recognize that a web engineering team must refine and adapt these generic tasks to the problem at hand and continue to refine them throughout the project.

Umbrella Activities

- Background activities which occur in parallel with the main development activities.
- Equally important to the success of a project and so should be considered explicitly.
- Many umbrella activities can be defined but only four are crucial for a successful web engineering project.

Umbrella Activities

- Change management: Manages the effects of change as each increment is engineered, integrating tools that assist in the management of all web application content.
- Quality assurance: Defines and conducts those tasks that help to ensure that each work product and the deployed increment exhibits quality.
- Risk management: Considers project and technical risks as an increment is engineered.
- Project management: Tracks and monitors progress as an increment is engineered.

“Communication” Phase

Week 3

Chapter 4: *Communication*

- *Understand the problem before you begin to solve it, and be sure that the solution you conceive is one that people really want.*
- To do this, you will need to:
  - *Formulate*
  - *Elicitate*
  - *Negotiate*

Formulation

- Focuses on defining the project needs and scope
  - begins with the identification of a business need
  - moves into a description of web application objectives
  - defines major web application features, and establishes a basis for the elicitation action that follows.
  - allows stakeholders and the web engineering team to establish a common set of goals and objectives for the creation of each web application increment.
  - identifies the scope of the development effort and provides a means for determining a successful outcome.

What questions do we ask?

- What is the main motivation (business need) for the web application?
- What are the objectives that the web application must fulfill?
- Who will use the web application?

Elicitation

- The intent is to gather detailed requirements collaboratively with all stakeholders.
- To do this:
  - A meeting (either physical or virtual) is conducted and attended by all stakeholders.
  - Rules for preparation and participation are established.
  - An agenda is suggested that is formal enough to cover all important points but informal enough to encourage the free flow of ideas.
  - A facilitator (can be a customer, a web engineer, or an outsider) controls the meeting.
  - A definition mechanism (can be work sheets, flip charts, or wall stickers or an electronic bulletin board, chat room, or virtual forum) is used.

Elicitation Tasks

- Define user categories and develop descriptions for each category.
- Define content and functionality using the lists each person prepared.
- Consider specific constraints and performance issues.
- Write user scenarios for each user class.

User Descriptions

- What is the user's overall objective when using the web application?
- What is the user's background and sophistication level relative to the content and functionality of the web application?
- How will the user arrive at the web application?
- What generic web application characteristics does the user like and dislike?

Content and Functionality

- Each stakeholder has begun this work by preparing lists of content objects and web application functions.
- Once the meeting begins these lists can be:
  - displayed on large sheets of paper pinned to the walls of the room
  - displayed on adhesive-backed sheets stuck to the walls, or written on a whiteboard.
  - posted on an electronic bulletin board, at an internal website, or posted in a chat room environment for review prior to the meeting.
- Ideally, each listed entry should be capable of being manipulated separately so that lists can be combined, entries can be deleted, and additions can be made. At this stage, critique and debate are strictly prohibited.

Constraints and Performance

- *Internal constraints* are best understood by thinking about the technical environment in which the web application will reside and the project environment in which the web application will be built.
  - ***Technical environment***—specialized database protocols, the vagaries of different web browsers, operating system characteristics, and client-server issues.
  - ***Project environment***—available web engineering tools, development hardware, software standards, and staff's skill levels with various web engineering technologies.

Constraints and Performance

- *External constraints* can be enumerated by considering the business and usage environment for the web application.
  - Business rules, end-user idiosyncrasies, security demands, privacy issues, run-time performance, interoperability requirements, legal restrictions, and government regulations are but a few of possible external constraints.

Capturing Interaction: Use Cases

- Use cases describe how a specific user category (called an *actor*) will interact with the web application to accomplish a specific action.
- Use cases are developed iteratively. Only those use cases necessary for the increment to be built are developed during the communication activity for the increment.
- Use cases enable you to:
    - provide the detail necessary for effective planning and modeling activities.
    - help you to understand how users perceive their interaction with the web application.
    - help to compartmentalize web engineering work because they can be organized into web application increments.
    - provide important guidance for those who must test the web application.

From Use Cases to Increments

- A stack of "cards" that contains one usage scenario or use case per card
  - Each card contains the name of the use case, a brief description, and an *effort indicator*—usually a number between 1 and 4.
- The cards are:
  - shuffled into random order
  - distributed to selected stakeholders who are asked to arrange the cards into groupings that reflect how they would like content and functionality (implied by the usage scenarios) to be delivered.
- The manner in which cards are grouped is constrained by an *effort maximum (M).*
  - No grouping of cards can have a cumulative effort indicator value that is greater than *M*, where *M* is defined by the web engineering team and is a function of available resources and the desired delivery time for each increment.

Negotiation

- Ideally, requirements are defined in sufficient detail to proceed.
- But in reality, requirements are often contradictory or infeasible (within the context of real-world constraints, such as cost or time).
- Negotiation involves working with the stakeholders to balance functionality, performance, and other product or system characteristics against cost and delivery time.
- The best negotiators strive for a win-win result.
  - it's a good idea to determine each of the stakeholders' "win conditions".

Negotiation

- *Recognize that it's not a competition:* To be successful, both parties have to feel they've won or achieved something. Both will have to compromise.

- *Map out a strategy:* Decide what you'd like to achieve, what the other party wants to achieve, and how you'll go about making both happen.

- *Listen actively:* Don't work on formulating your response while the other party is talking and listen actively. It's likely you'll gain knowledge that will help you to better negotiate your position.

Negotiation

- *Focus on the other party's interests:* Don't take hard positions if you want to avoid conflict.
- *Don't let it get personal:* Focus on the problem that needs to be solved.
- *Be creative:* Don't be afraid to think outside of the box if you're at an impasse.
- *Be ready to commit:* Once an agreement has been reached, don't waffle; commit to it and move on.

“Modeling” Phase

Week 4

Chapter 6: *The Modeling Activity*

- Analysis modeling helps you to understand the nature of the problem being addressed and the "shape" of the web application that will allow you to address that problem.
- Design modeling is about understanding the internal structure of the web application being developed and how this creates the shape of the web application that was identified by the analysis model.

Modeling Languages

- A *modeling language* (ML) incorporates a set of notations, terms, and/or symbols as well as the rules for establishing associations between them.
- A *modeling language* often has a formally structured representation as well as a set of graphical elements.
- Some MLs are general purpose (e.g., UML) and others are more specific (e.g., WebML)

Modeling Languages

- What capabilities should exist to model functionality?
  - Ability to model integration and connectivity.
  - Ability to support pattern modeling.
  - Ability to represent concepts in a technology-neutral fashion.
  - Ability to model sophisticated system functionality.

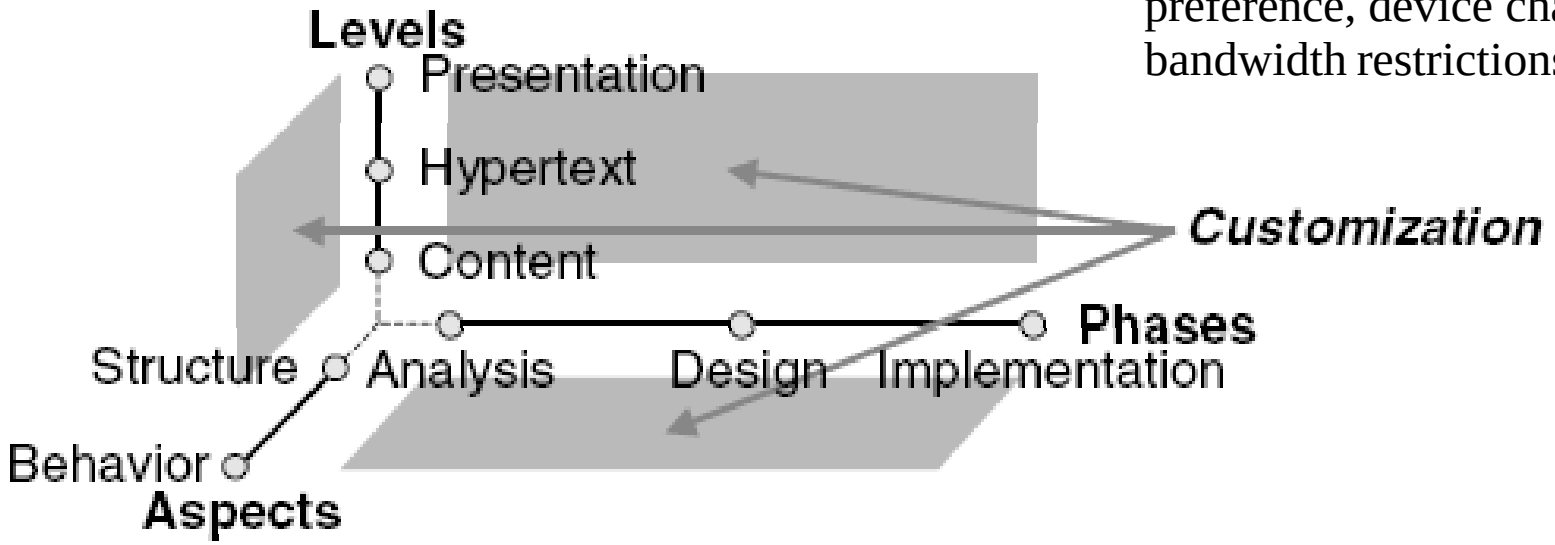
- What capabilities should exist to model information content?
  - Ability to model presentation-level concepts.
  - Ability to model navigational structure and behavior.
  - Ability to model user interactions with the information.
  - Ability to model user roles and user groups.
  - Ability to model content.

- What generic capabilities should exist in a modeling language?
  - Ability to model business domain concepts.
  - Ability to link business models with the technical architecture.
  - Ability to link information with functionality.
  - Ability to maintain system integrity.
  - Ability to support understanding and communication.
  - Ability to support Web system life cycle management.

Web Application Modeling

Content = information + application logic (hypertext-oriented vs. presentation-oriented)

Consider the context: user's preference, device characteristics, bandwidth restrictions, etc.



Static information driven vs. high interaction driven

References

- www.csun.edu/~twang/595WEB/Slides/WebAppsModeling.pdf
- www.csun.edu/~twang/595WEB/Slides/UWE.pdf
- www.csun.edu/~twang/595WEB/Slides/WebML.pdf

Analysis Modeling

Week 5

Chapter-7: Analysis Modeling

- Analysis modeling helps you to understand the detailed requirements that will allow you to satisfy user needs.

- Analysis models look at content, interaction, function and behavior, and the web application configuration.

- To determine the how much analysis modeling to do, examine the:
    - Size and complexity of the web application increment
    - Number of stakeholders (analysis can help to identify conflicting requirements coming from different sources).
    - Size of the web engineering team
    - Degree to which members of the web engineering team have worked together before analysis, can help to develop a common understanding of the project.
    - Degree to which the organization's success is directly dependent on the success of the web application.

Analysis Outputs

- **Interaction model**: Describes the manner in which users interact with the web application.
- **Information model:** Identifies the full spectrum of content to be provided by the web application. Content includes text, graphics and images, and video and audio data.
- **Functional model**: Defines the operations that will be applied to web application content and describes other processing functions that are independent of content but necessary to the end user.
- **Configuration model**: Describes the environment and infrastructure in which the web application resides.

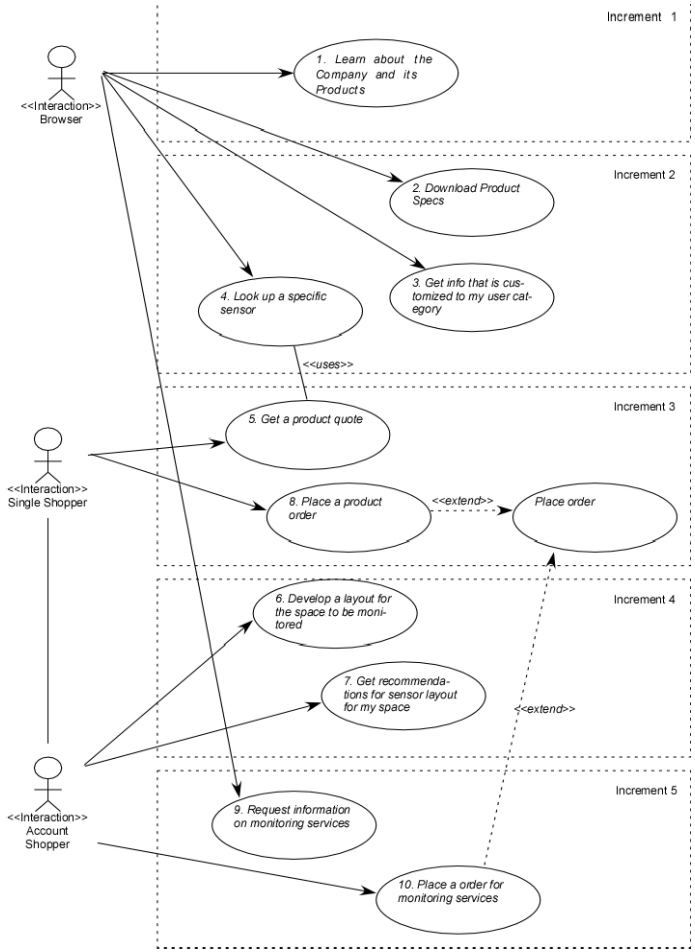
Understanding Users

- Crucial to understand your users!
- For each user class:
  - What is the user's overall objective?
  - What is the user's background?
  - How will the user arrive at the web application?
  - What characteristics does the user like and dislike?

Revisiting Use Cases

- Analyze and elaborate where necessary
- Find gaps, missing details

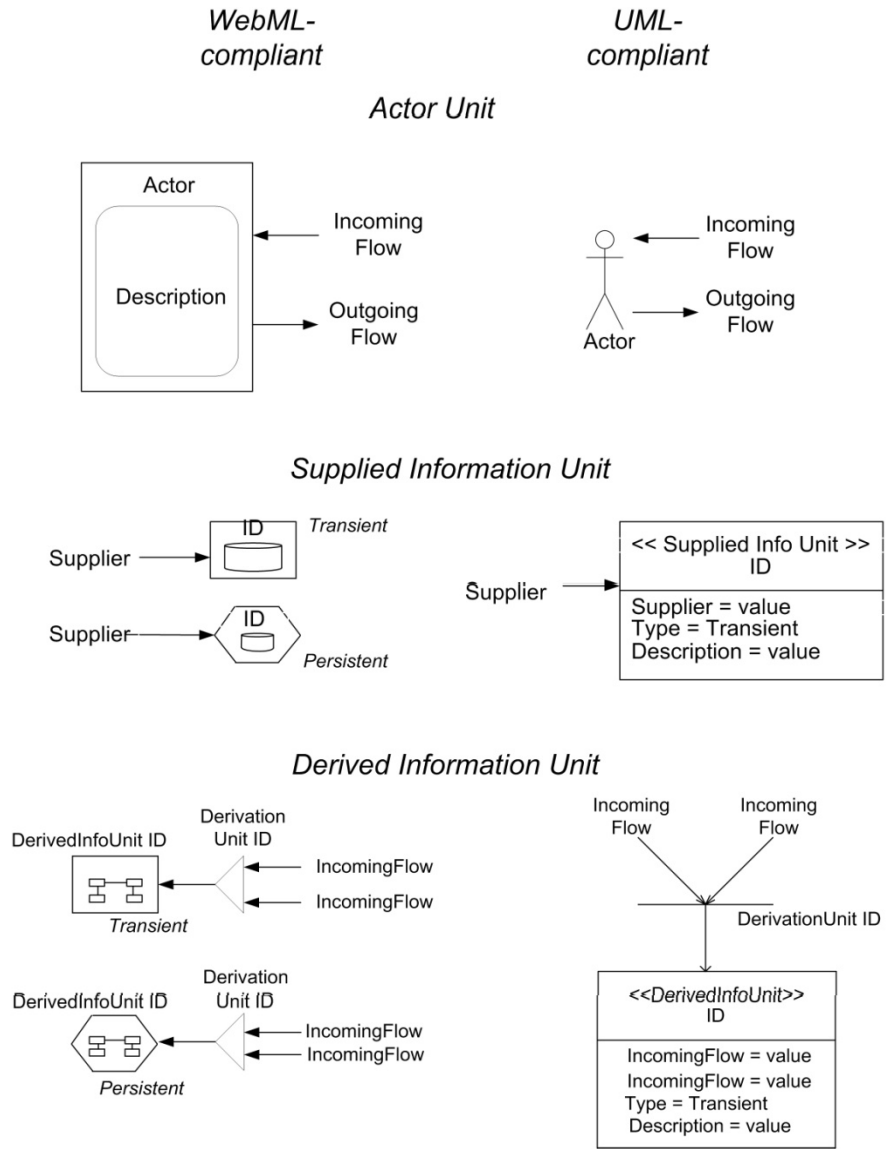
- Identify overlaps and possible optimizations
  - Allows design simplification
  - E.g. often “view” task can be seen as a specialization of an “edit” task.



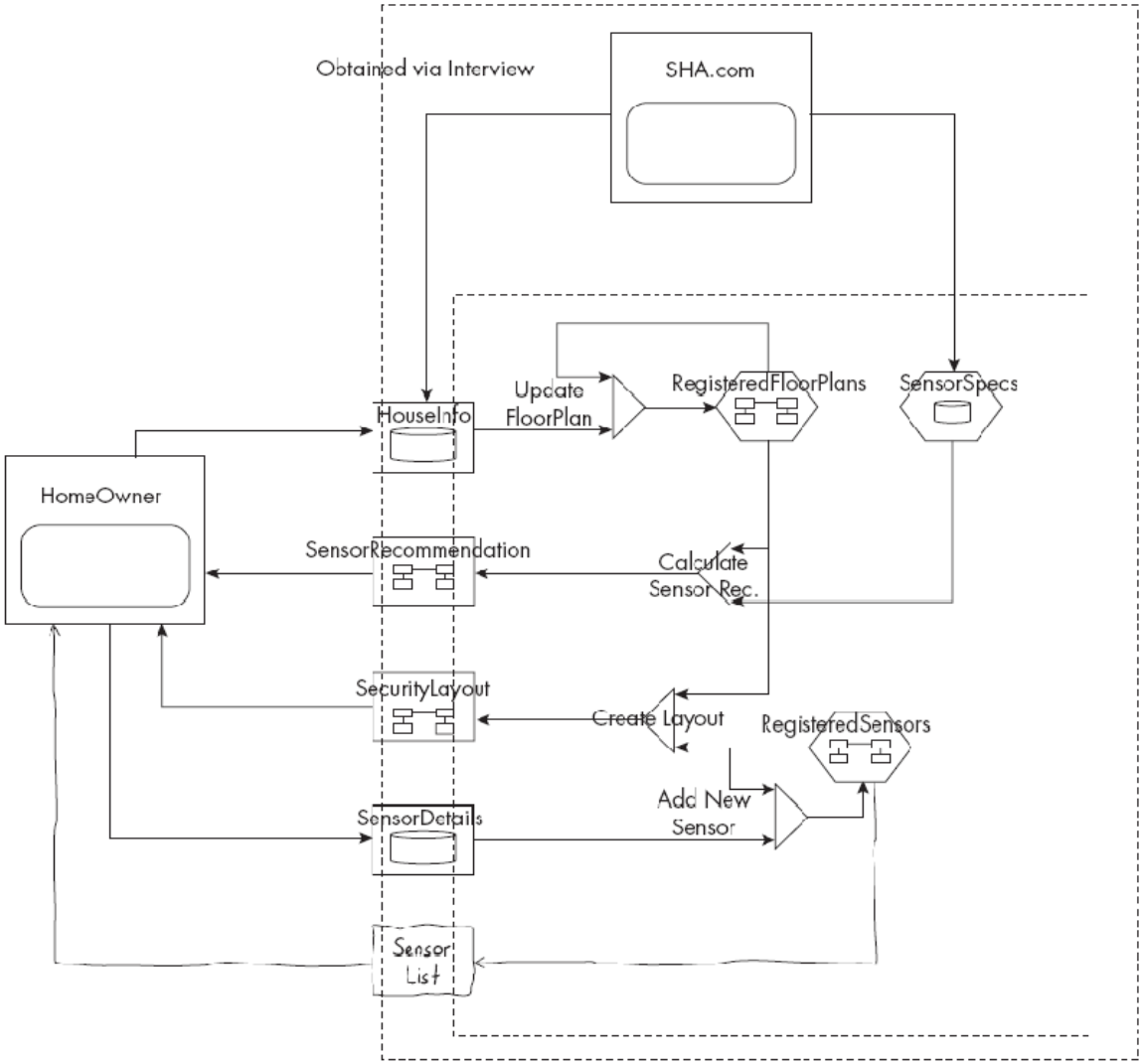
The Content Model

- Identify content objects:
  - *External entities* (e.g., other systems, databases, people) that produce or consume information to be used by the web application.
  - *Things* (e.g., reports, displays, video images) that are part of the information domain for the problem
  - *Occurrences or events* (e.g., a quote or an order) that occur within the context of a user's interaction with a web application.
  - *Roles* (e.g., retail purchasers, customer support, salesperson) played by people who interact with the web application.
  - *Organizational units* (e.g., division, group, team) that are relevant to an application
  - *Places* (e.g., manufacturing floor or loading dock) that establish the context of the problem and the overall function of the web application.
  - *Structures* (e.g., sensors, monitoring devices) that define a class of objects or related classes of objects

Web Info. Exchange - Notation

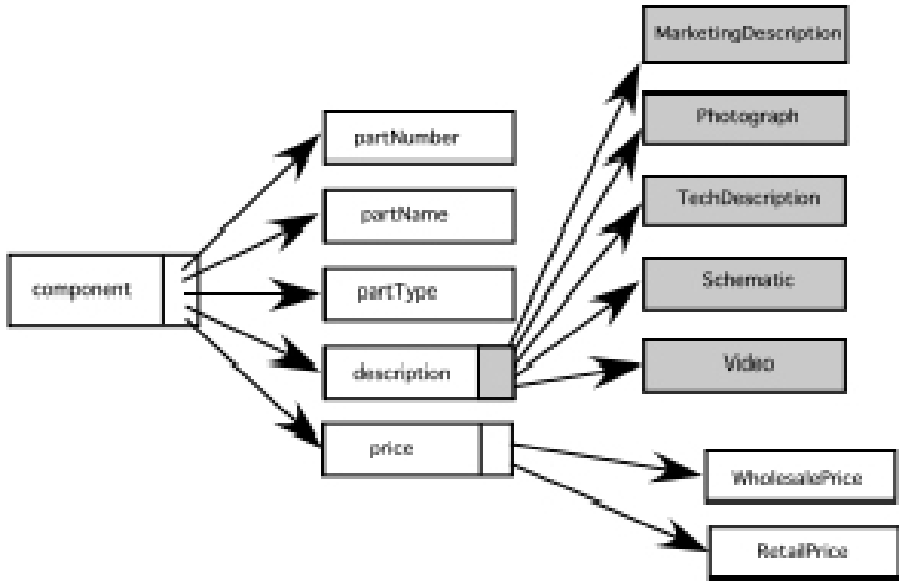


Web Info. Exchange - Example



Data Tree

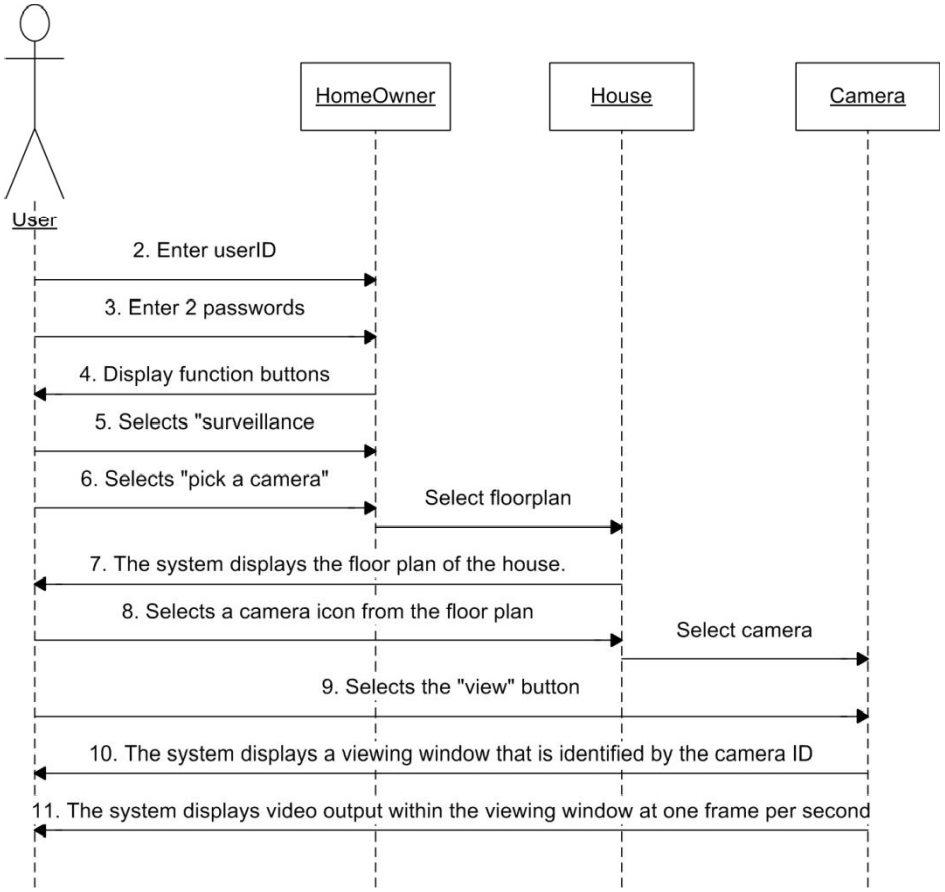
- In some cases, the content model may benefit from a richer analysis
- *Data trees* depict the relationships among content objects and the hierarchy of content maintained by a web application.



The Interaction Model

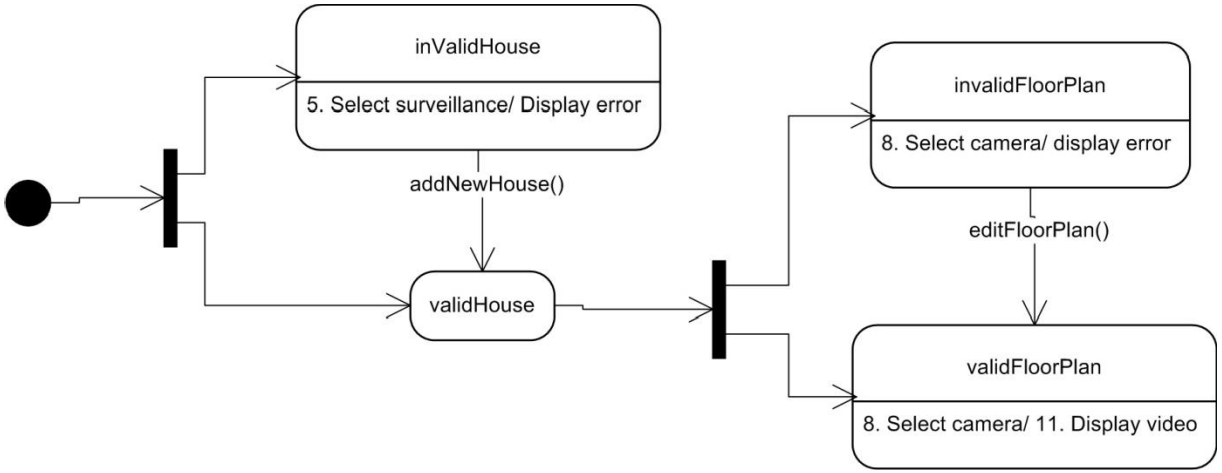
- The interaction model can be represented using:
  - Use cases
  - Sequence diagrams
  - State diagrams
  - User interface prototypes
- In many instances, a set of use cases is sufficient to describe the interaction at an analysis level (further refinement and detail will be introduced during design).
- However, when the sequence of interaction is complex and involves multiple analysis classes or many tasks, it is sometimes worthwhile to depict it using a more rigorous diagrammatic form.

Sequence Diagram



UML *sequence diagrams* describe how user actions collaborate with analysis classes (the structural elements of a system).

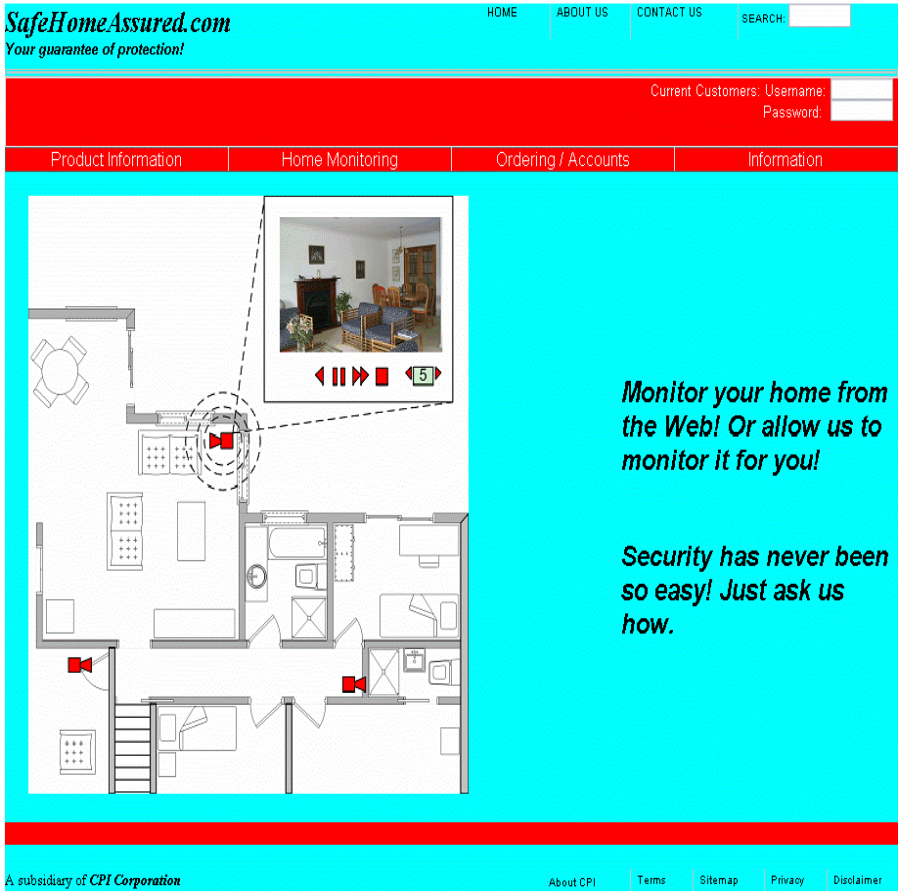
State Diagram



- UML *state diagrams* describe dynamic behavior of the web application as an interaction occurs.

- State diagrams are most useful when a user interaction triggers a change in the state of the web application—and hence changes the way in which it might react to a user.

Active Interface Prototype



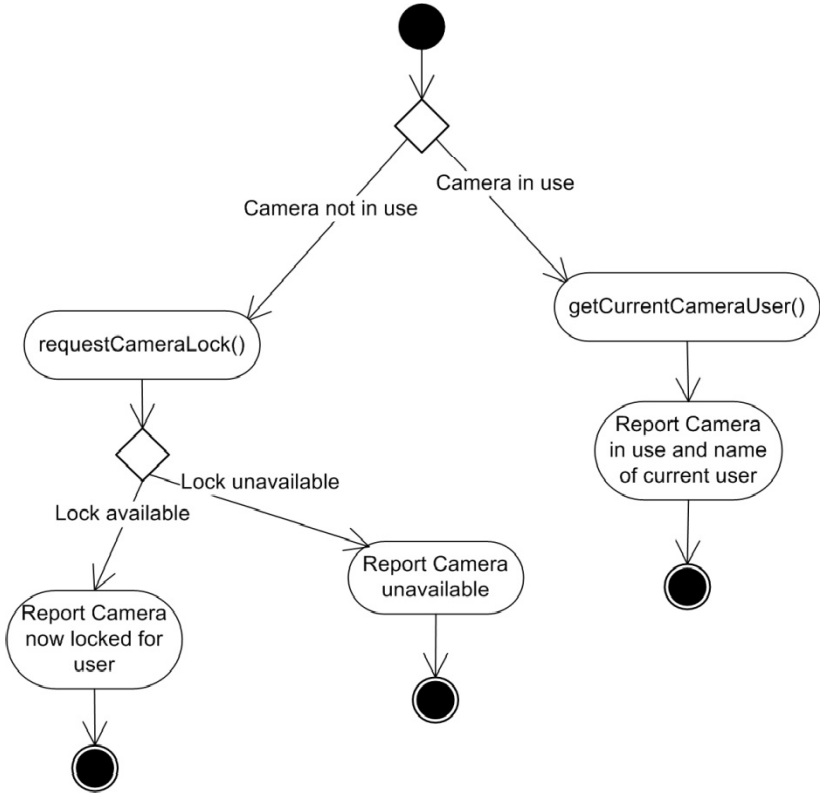
- A prototype shows the layout of the user interface, the content, interaction mechanisms and overall aesthetics.
- Supports validation with the client of the requirements and analysis.

The Functional Model

- Addresses two processing elements of the web application, each representing a different level of procedural abstraction: – user-observable functionality that is delivered by the web application to end users, and
  - the operations contained within analysis classes that implement behaviors associated with the class.
- The UML activity diagram can be used to represent processing details.

Activity Diagram

- Illustrates the processing flow and logical decisions within the flow.
  - The construction details indicate how these operations are invoked, and the interface details for each operation are not considered until web application design commences.



The Configuration Model

- Among the many configuration issues that should be addressed are:
  - Server hardware and operating system environments
  - Interoperability considerations on the server side (e.g., large database access, other IT applications, specialized communication protocols)
  - On the client side:
    - Local OS
    - Browser software
    - Client hardware variations

Web Design

Week 6

Web App Design-1

- Jakob Nielsen states: “There are essentially two basic approaches to design: the artistic ideal of expressing yourself and the engineering ideal of solving a problem for a customer.”

Web App Design – II

- Even today, some proponents of agile software development use Web Apps as poster children for the development of applications based on “limited design.”
- However --
  - when contents and functions are complex
  - when the size of the Web App encompasses hundreds of content, objects, functions, and analysis classes
  - when multiple people become involved in the design
  - when the success of the Web App will have a direct impact on the success of the business.
- Design cannot and should not be taken lightly.

Web App Design

- The design model encompasses content, aesthetics, architecture, interface, navigation, and component-level design issues.

- The design model provides sufficient information for the web engineering team to construct the final web application.

- The current design model will lead to an effective implementation is also assessed.

Design Goals - I

- **Simplicity:** Although it may seem old-fashioned, the aphorism "all things in moderation" applies to web applications. Rather than *feature-bloat*, it is better to strive for moderation and simplicity.

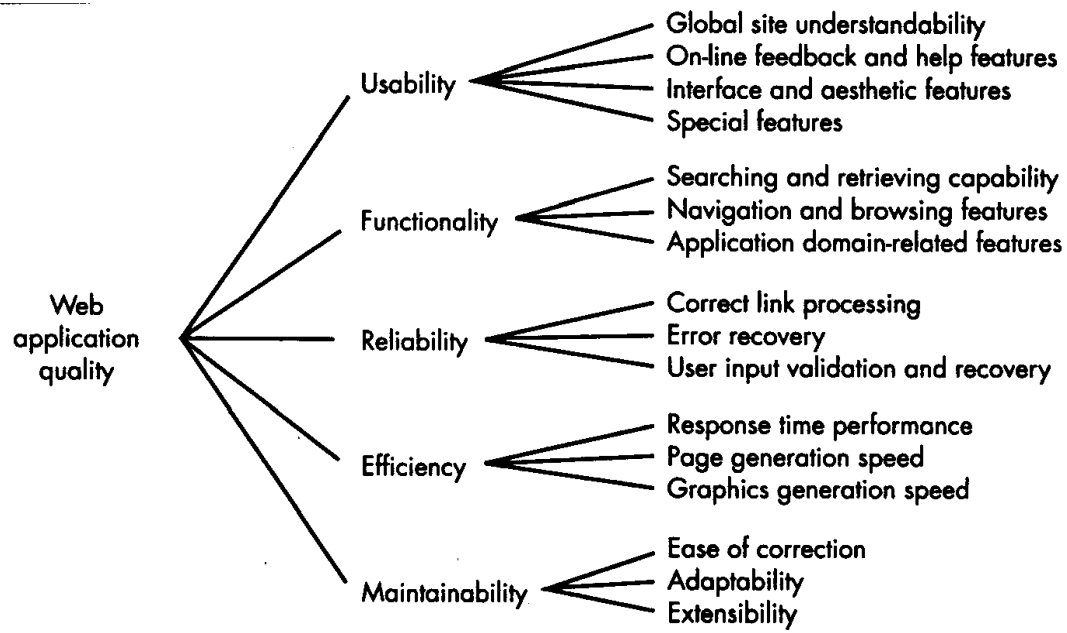
- **Consistency:**
  - Content should be constructed consistently
  - Graphic design (aesthetics) should present a consistent look
  - Architectural design should establish templates that lead to a consistent hypermedia navigation
  - Navigation mechanisms should be used consistently

- **Identity:** The aesthetic, interface, and navigational design of a web application must be consistent with the application domain for which it is to be built.

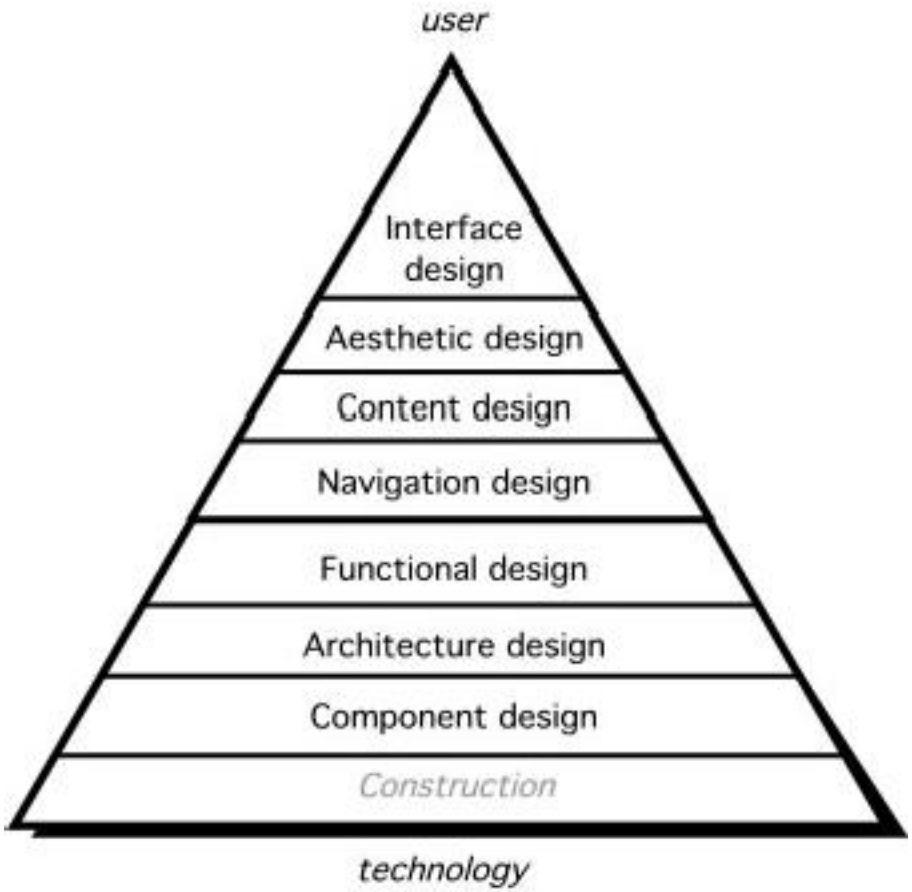
Design Goals - II

- **Robustness:** The user expects robust content and functions that are relevant to the user's needs.
- **Navigability:** Users should be able to understand how to move about the Web App without having to search for navigation links or instructions.
- **Visual appeal:** Design characteristics (e.g., the look and feel of content, interface layout, color coordination, the balance of text, graphics and other media, and navigation mechanisms) contribute to visual appeal.
- **Compatibility:** Most Web Apps will be used in a variety of environments (e.g., different hardware, Internet connection types, operating systems, and browsers) and must be designed to be compatible with each.

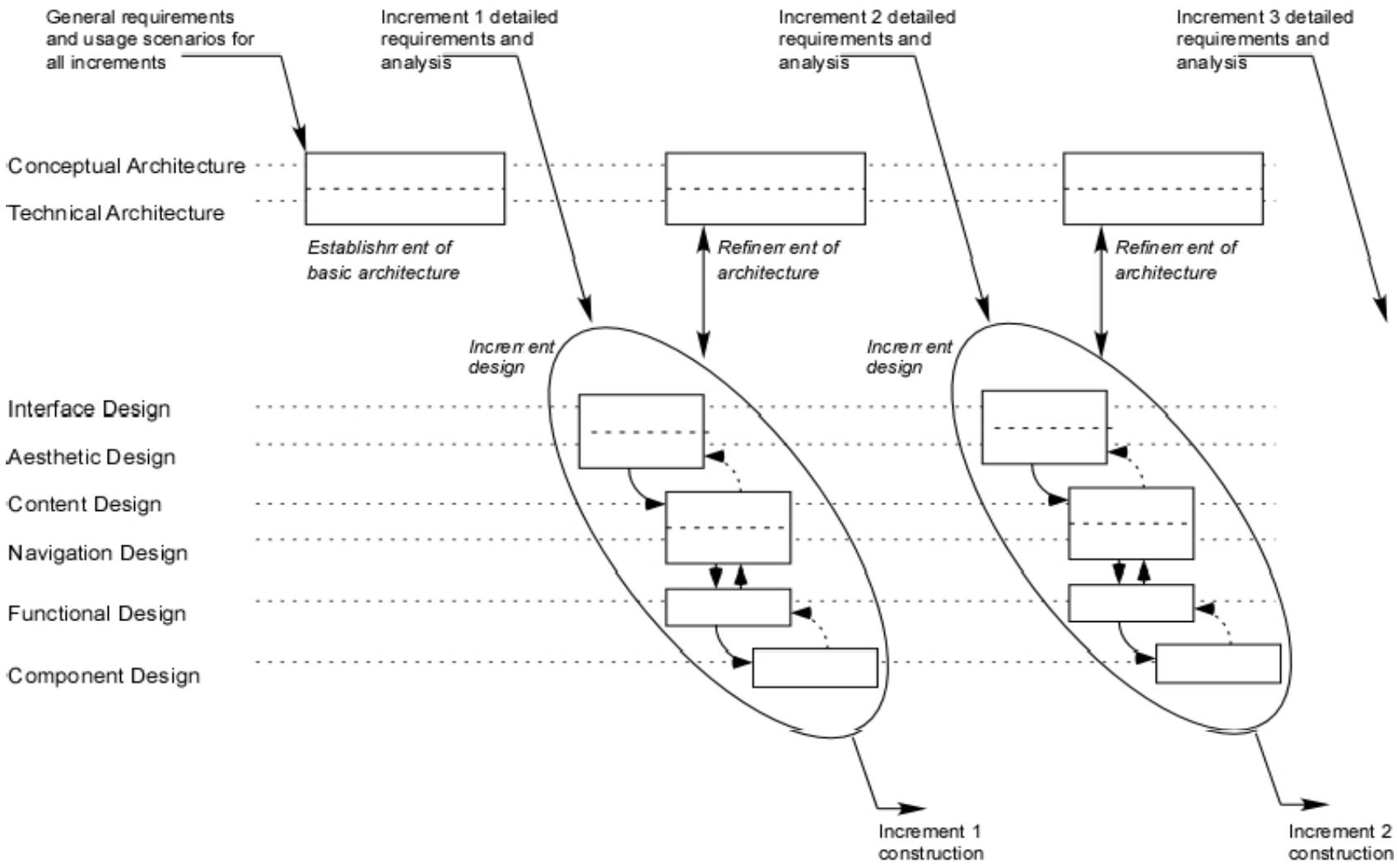
Design & web application quality



Design Actions



Design Process



Conceptual Architecture

- Provides an overall structure for the web application design
  - Affects all later increments – so important for it to developed in the context of the full set of *likely* increments.
- Represents the major functional and information components for the web application and describes how these will fit together.
  - Depends on the nature of the web application, but in every case, it should ensure a sound integration between the web application information and the web application functionality.

Developing the architecture-I

- How do we achieve an effective balance between information and functionality in the conceptual architecture?
- A good place to start is with workflows or functional scenarios (which are an expression of the system functionality) and information flows.

Developing the architecture-II

- As a simple example, consider the following set of key functionalities for **SafeHomeAssured.com**
  - Provide product quotation
  - Process security system order
  - Process user data
  - Create user profile
  - Draw user space layout
  - Recommend security system for layout
  - Process monitoring order
  - Get and display account info
  - Get and display monitoring info
  - Customer service functions (to be defined later)
  - Tech support functions (to be defined later)

Developing the architecture-III

- From these key functionalities we can identify the following partial list of *functional subsystems:*
  - **User Management:** Manages all user functions, including user registration, authentication and profiling, user-specific content, and interface adaptation and customization.
  - **Product Management**: Handles all product information, including pricing models and content management.
  - **Order Handling**: Supports the management of customers' orders.
  - **Account Administration**: Manages customers' accounts, including invoicing and payment.
  - **Security System Support**: Manages users' space layout models and recommends security layouts.
  - **Security System Monitoring**: Monitors customers' security systems and handles security events.

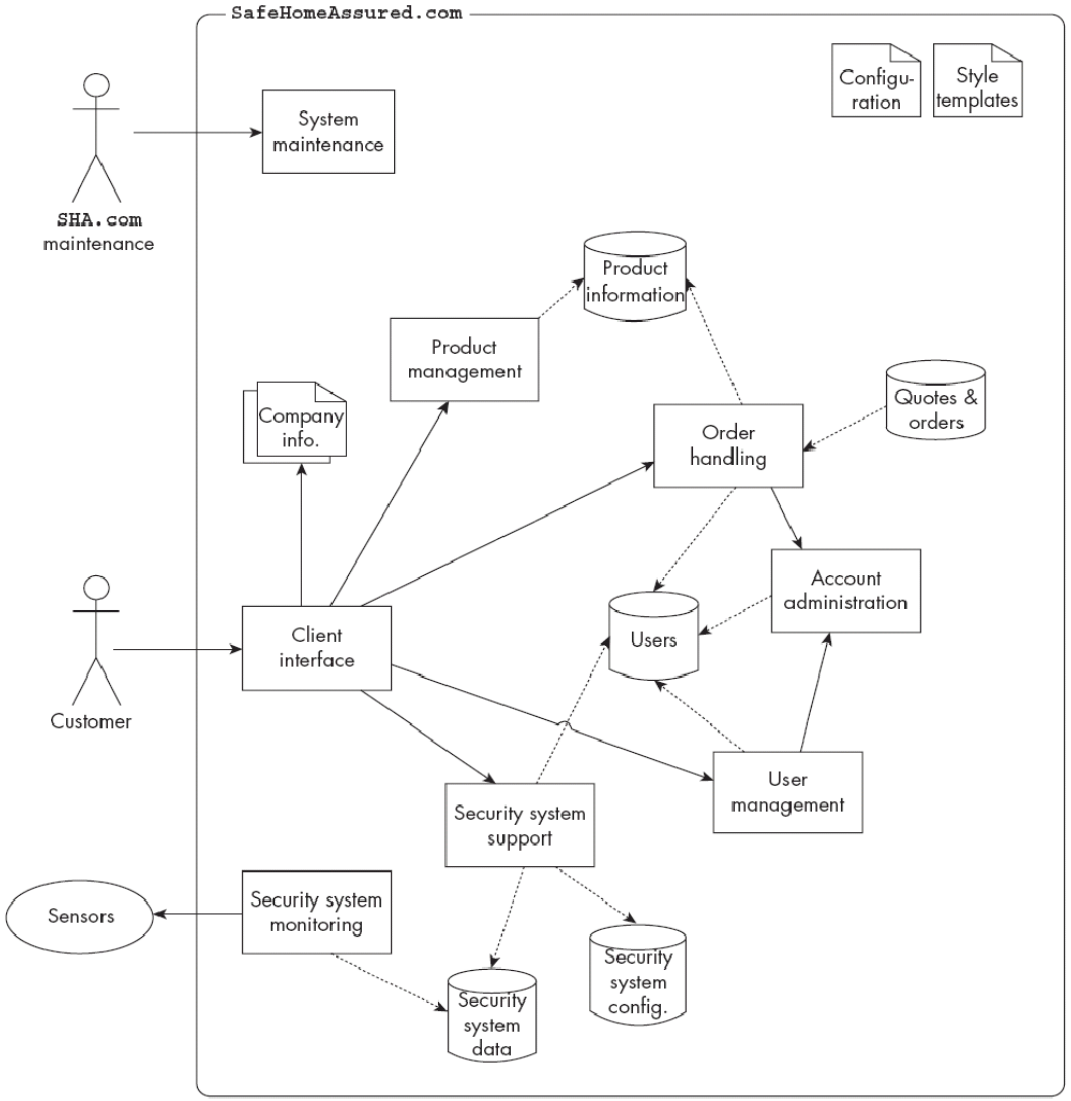
Developing the architecture-IV

- And, of course, there are overall management subsystems:
  - **Client Interface**: Provides the interface between users and the other subsystems, as required to satisfy user's needs.
  - **System Maintenance**: Provides maintenance functionality, such as database cleaning.

Technical Architecture

- Shows how the conceptual architecture can be mapped into specific technical components
- Any decision made about how one component might map into the technical architecture will affect the decisions about other components
  - For example, the web engineering team may choose to design **SafeHomeAssured.com** in a way that stores product information as XML files. Later, the team discovers that the content management system doesn't easily support access to XML content, but rather assumes that the content will be stored in a conventional relational database. One component of the technical architecture conflicts with constraints imposed by another component.

Architecture



Interface Design

Week 7

Interaction Design

- Design an interface to answer three generic questions:
  - *Where am I?* The interface should provide an indication of the web application that has been accessed and inform users of their current location in the content hierarchy.
  - *What can I do now?* The interface should always help users to understand their current options—what functions are available, what links are live, what content is relevant?
  - *Where am I going?* The interface must facilitate navigation. Hence, it must provide a "map" (implemented in a way that is easy to understand) of where users have been and what paths they may take to move elsewhere within the web application.

Design Principles - I

- ***Anticipation:*** *Design interface so that it anticipates the user's next move.*
- ***Communication:*** *The interface should communicate the status of any activity initiated by the user.*
- ***Consistency:***
  The use of navigation controls, menus, icons, and aesthetics (e.g.color, shape, layout) should be consistent throughout the web application.
- ***Controlled autonomy:*** *The interface should facilitate user movement throughout the web application, but it should do so in a manner that enforces navigation conventions that have been established for the application.*
- ***Efficiency:*** *The design of the web application and its interface should optimize the user's work efficiency, not the efficiency of the Web engineer who designs and builds it or the client-server environment that executes it.*
- ***Flexibility:*** *The interface should be flexible enough to enable some users to accomplish tasks directly and others to explore the web application in a somewhat random fashion.*
- ***Focus:*** *The web application interface and the content it presents should stay focused on the user tasks at hand.*

Design Principles-II

- ***Fitt's law:*** *"The time to acquire a target is a function of the distance to and size of the target"*
- ***User Interface Objects.*** *A vast library of reusable human interface objects (and patterns) has been developed for web application.*
- ***Latency reduction.*** *Rather than making the user wait for some internal operation to complete (e.g., downloading a complex graphical image), the web application should use multitasking in a way that lets the user proceed with work as if the operation has been completed.*
- ***Learnability.*** *A web application interface should be designed to minimize learning time and to minimize relearning required when the web application is revisited.*
- ***Metaphors.*** *An interface that uses an interaction metaphor is easier to learn and easier to use, as long as the metaphor is appropriate for the application and the user.*

Design Principles-III

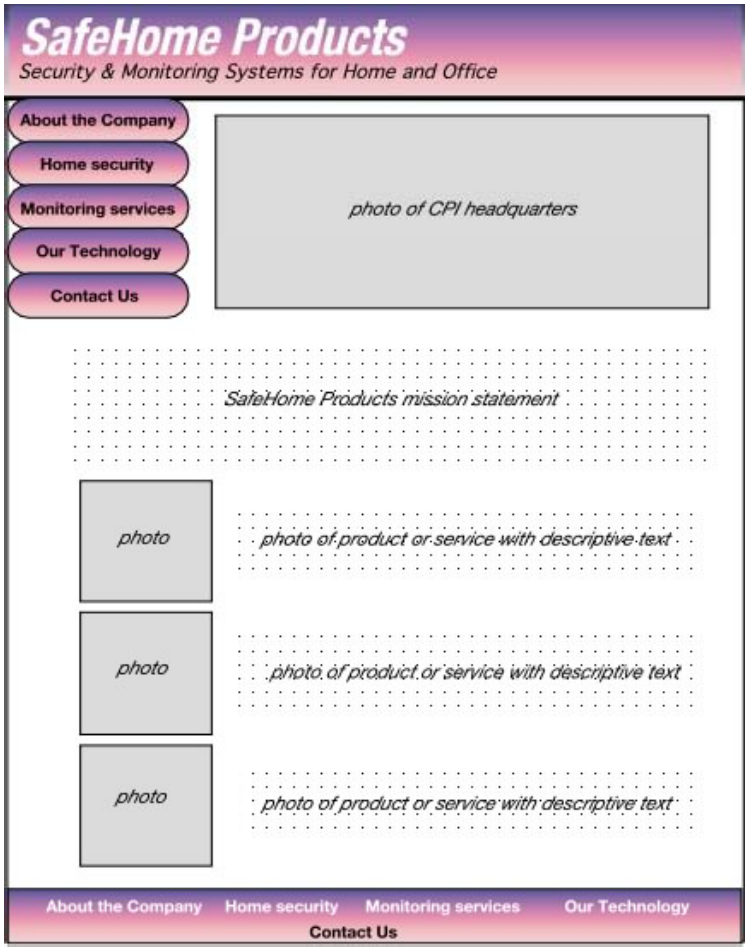
- ***Maintain work product integrity:*** *A work product (e.g., a form completed by the user, a user-specified list) must be automatically saved so that it will not be lost if an error occurs.*

- ***Readability:*** *All information presented through the interface should be readable by young and old.*

- ***Track state:*** *When appropriate, the state of user interactions should be tracked and stored so that users can log off and return later to pick up where they left off.*

- ***Visible navigation:*** *A well-designed web application interface provides "the illusion that users are in the same place, with the work brought to them"*

Preliminary Page Layout

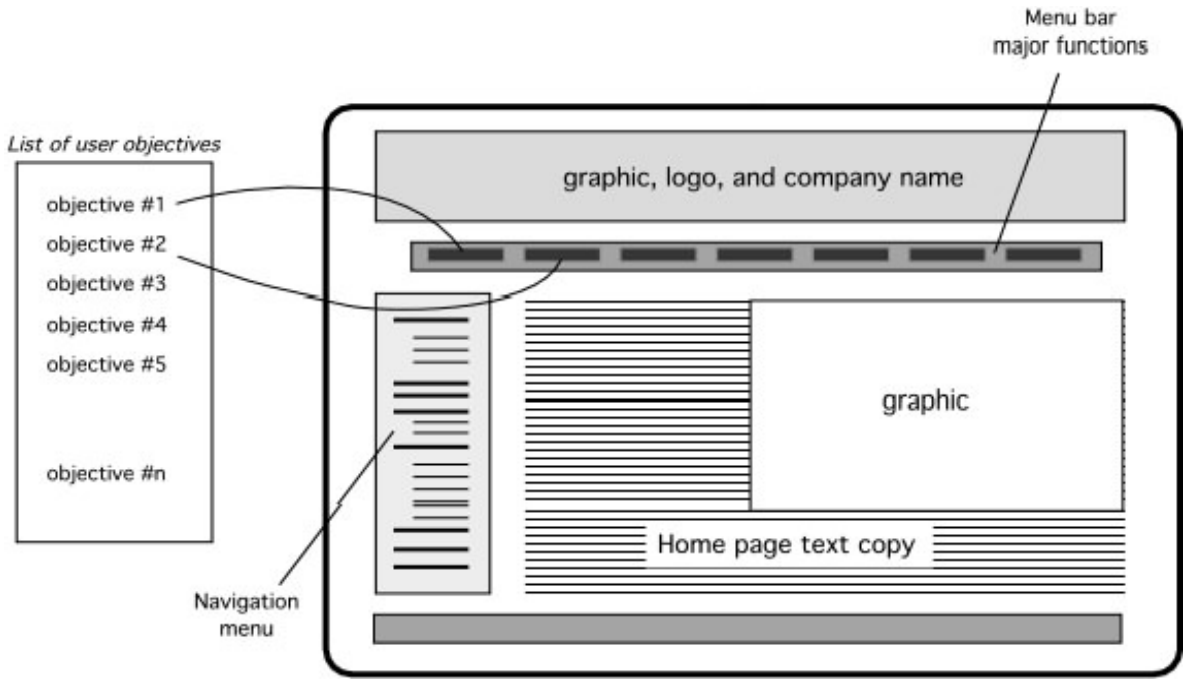


Pragmatic Design Guidelines

- Reading speed on a computer monitor is approximately 25 percent slower than reading speed for hard copy. Therefore, *do not force the user to read voluminous amounts of text.*
- *Avoid "under construction" signs* —they raise expectations and cause an unnecessary link that is sure to disappoint or frustrate users.
- *Users prefer not to scroll.* Important information should be placed within the dimensions of a typical browser window.
- *Navigation menus and head bars should be designed consistently and should be available on all pages that are available to the user.* The design should not rely on browser functions (e.g., the back arrow) to assist in navigation.
- *Aesthetics should never supersede functionality.* For example, a simple button might be a better navigation option than an aesthetically pleasing, but vague image or icon whose intent is unclear.
- *Navigation options should be obvious, even to the casual user.* The user shouldn't have to search the screen to determine how to link to other content or services.

Interface Design Workflow - I

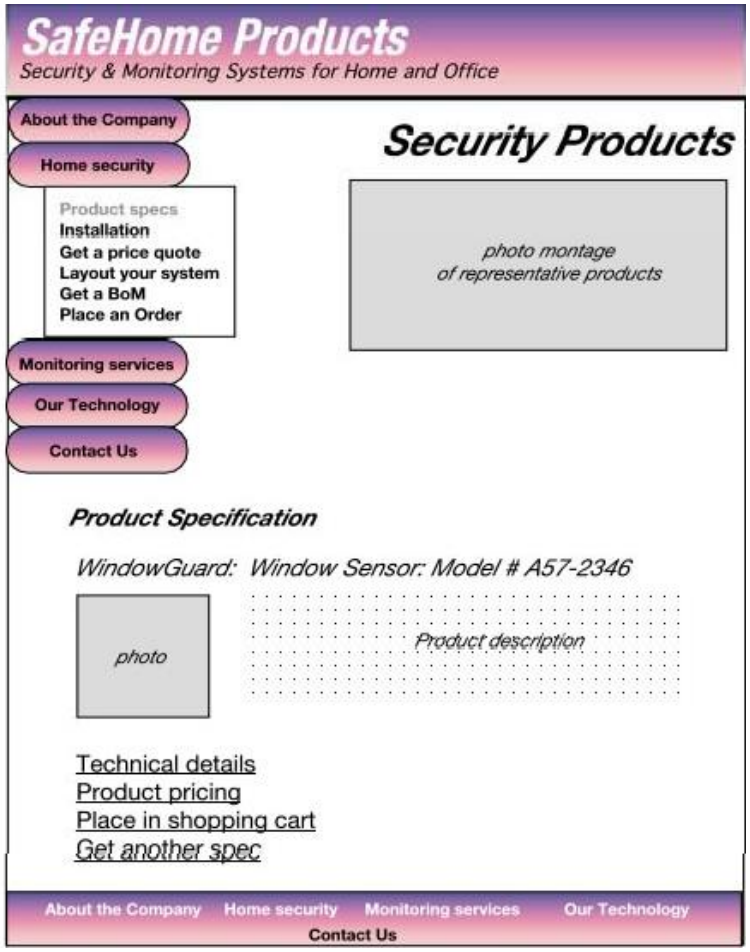
- Review user characteristics and categories, user tasks, use cases, and related information contained in the analysis model and refine as required.
- Develop a rough design prototype of the web application interface layout.
- Map user objectives into specific interface actions.



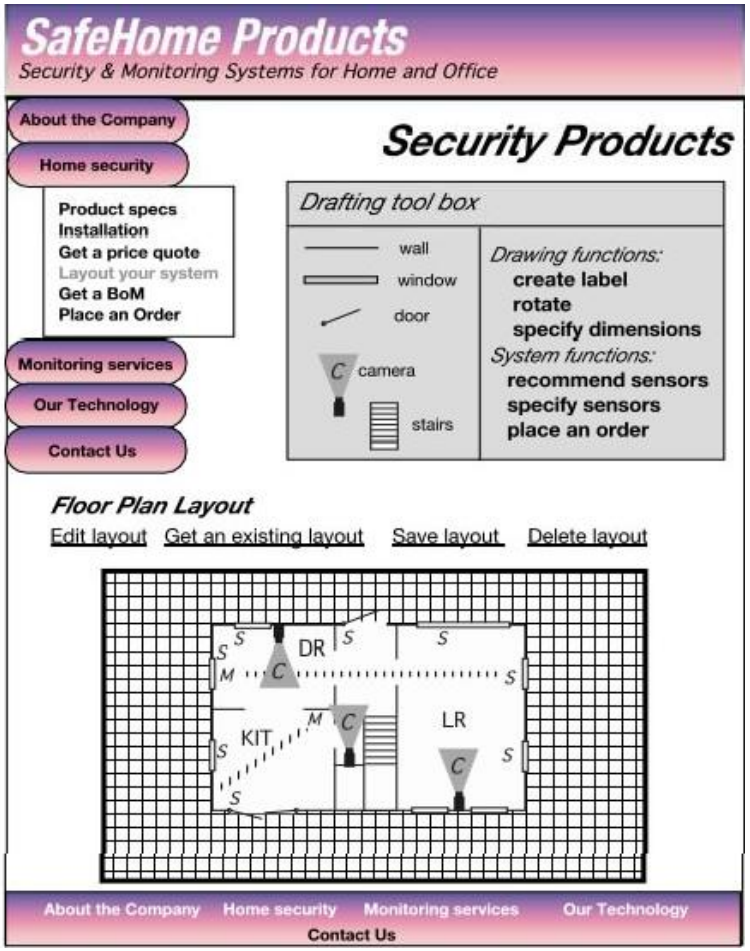
Interface Design Workflow - II

- Define a set of user tasks that are associated with each action.
- Develop screen images for each interface action.
- Refine interface layout and screen images using input from aesthetic design.
- Identify user interface objects that are required to implement the interface.
- Develop a procedural representation of the user's interaction with the interface.
- Develop a behavioral representation of the interface.
- Describe the interface layout for each state.
- Pair walkthroughs (Chapter-5) should be conducted throughout all these design tasks and should focus on usability.

Elaborate the design



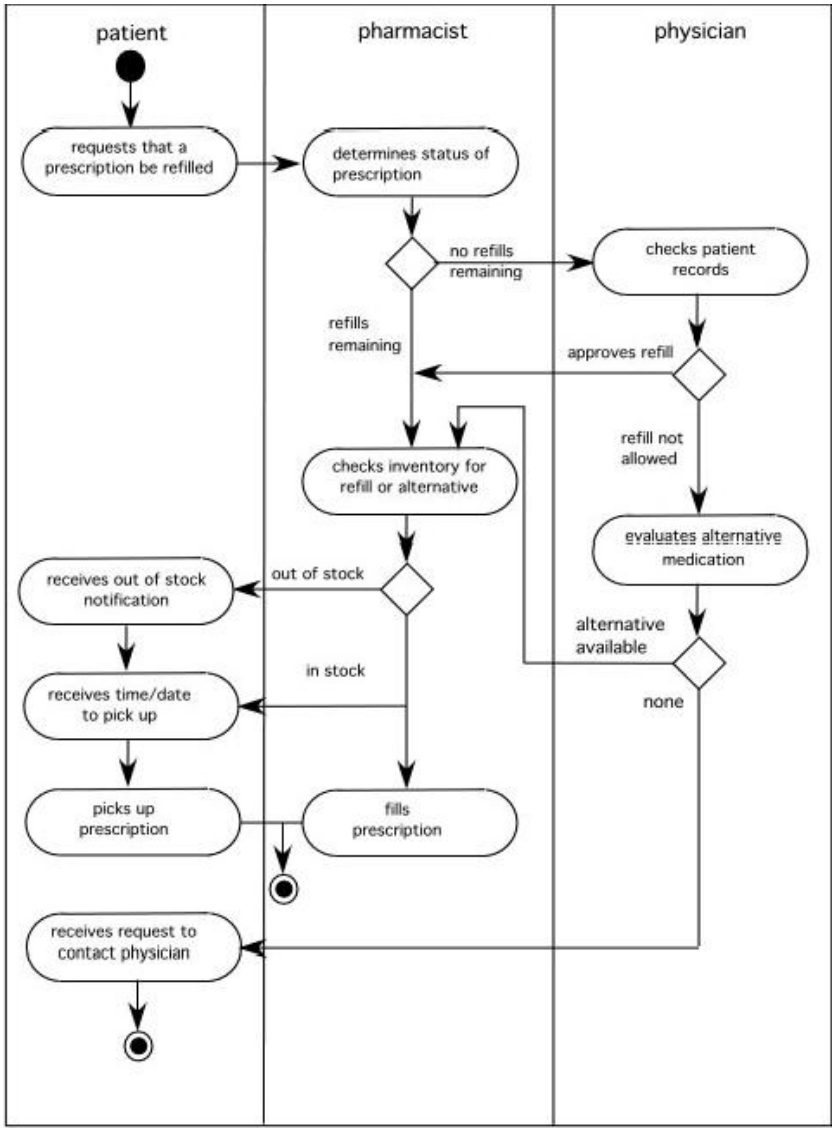
Elaborate the Design



Different Users in Different Roles

The swimlane diagram:

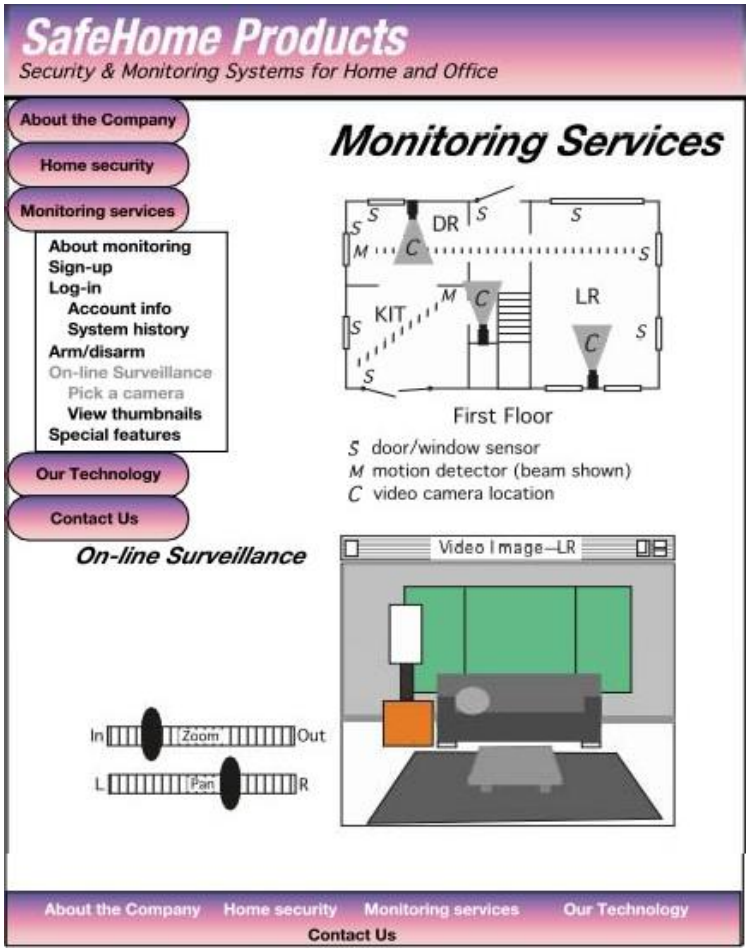
Captures workflows and shows interactions between different users



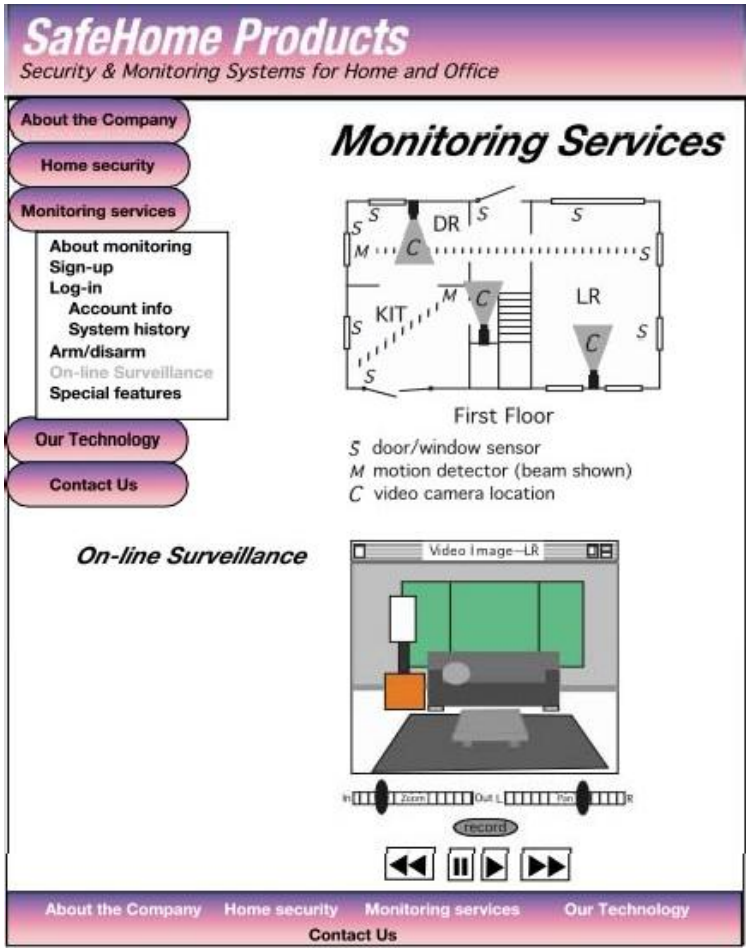
Translating Actions and Objects

- From the use case on pp. 213 - 214
  - *Accesses* the **SafeHome** system
  - *Enters* an **ID** and **Password** to allow remote access
  - *Displays* **FloorPlan** and **SensorLocations**
  - *Displays* **VideoCameraLocations** on floor plan
  - *Selects* **VideoCamera** for viewing
  - *Views* **VideoImages** (four frames per second)
  - *Pans* or *zooms* the **VideoCamera**
- Based on these actions and objects we create a design layout :

Design Layout



Revising the Layout



Aesthetic Design

- Don’t be afraid of white space.
- Emphasize to all content equally.
- Organize layout elements from top-left to bottom-right.
- Don’t extend your real estate with the scrolling bar.
- Consider resolution and browser window size when designing the layout.
- Design the layout for freedom of navigation.
- Don’t assume that the layout will be consistent across different display devices and browsers.
- If you use photos, make them small format with the option to enlarge.
- If you want a cohesive layout, look, and feel across all web application pages, use a cascading style sheet (CSS).

Usability

- Is the web application usable without continual help or instruction?
- Do the rules of interaction and navigation help a knowledgeable user work efficiently?

  Do interaction and navigation mechanisms become more flexible as users become more knowledgeable?
- Has the web application been tuned to the physical and social environment in which it will be used?
- Are users aware of the state of the web application? Do users know where they are at all times?
- Is the interface structured in a logical and consistent manner?
- Are interaction and navigation mechanisms, icons, and procedures consistent across the interface?
- Does the interaction anticipate errors and help users correct them?
- Is the interface tolerant of errors that are made?
- Is the interaction simple?

Other Design Issues

- Response time
- “Help” facilities
- Error handling
- Accessibility
- Internationalization

Information Design

Week 8

Chapter 10 - *Information Design*

- Three key issues:
  - **Content:** What content is available?
  - **Composition:** What views on that content do we wish to provide users?
  - **Navigation:** How do the users gain access to those views?
- There are different levels of abstraction at which we might consider these information design issues.
- Information design addresses *how content can be organized, accessed, and managed.*

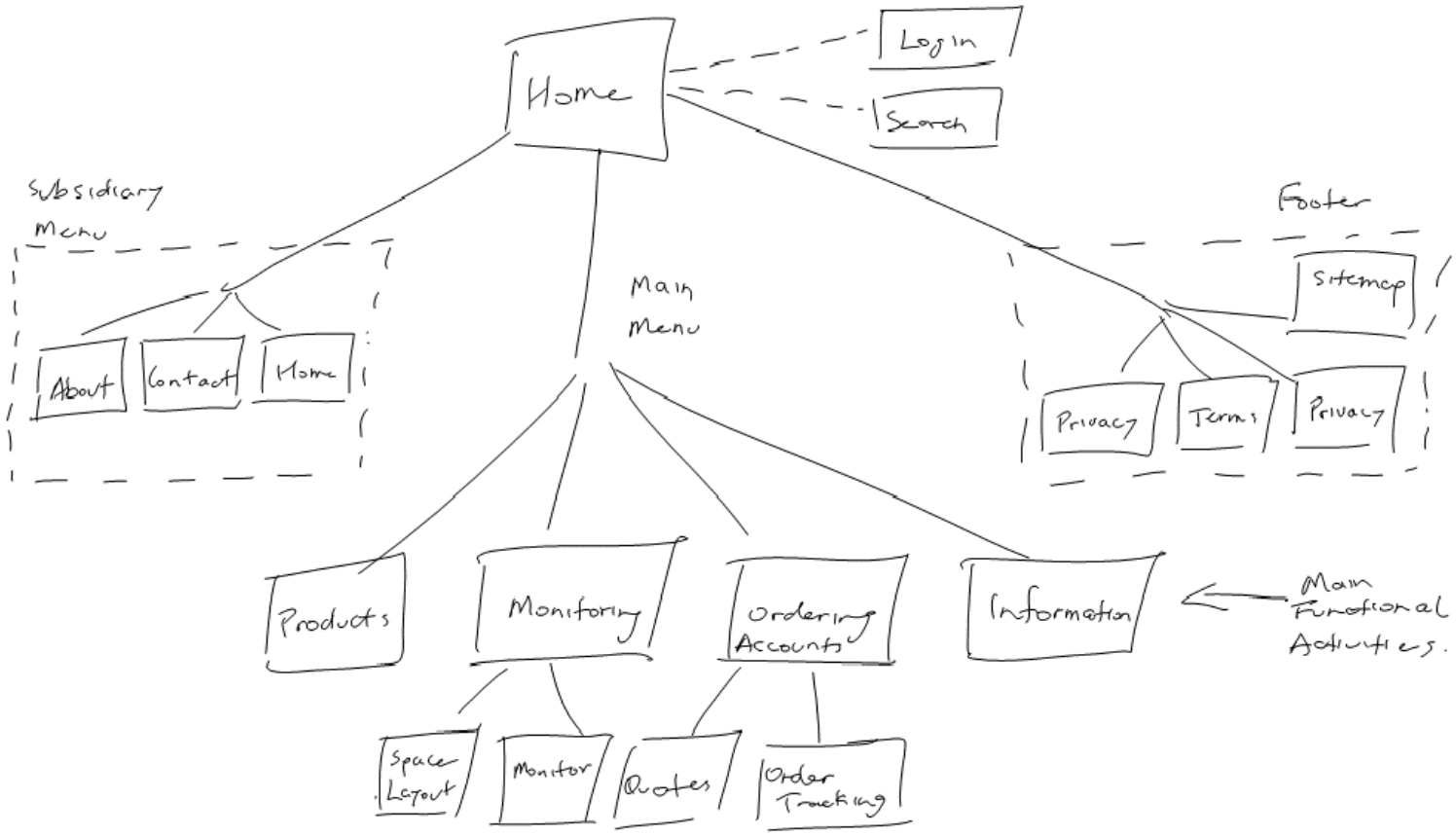
Information Architecture (IA)

- The overall strategy for information design usually combines both bottom-up and top-down approaches:
  - **Bottom-up:** Commonly used for small web applications; Build pages and progressively link them into the structure.
  - **Top-down:** Considers overall organization – the realm of the *Information Architect.*
- *"The structural design of an information space to facilitate task completion and intuitive access to content".*

Information Architecture (IA)

- As simple as a site map that represents the basic web application navigational structure.
- May be a detailed model that provides a comprehensive overview of the approach for structuring, managing, and accessing information within a web application.
- Provides a skeleton around which all information aspects of the web application are built:
  - Describe the basic information “structure” of the solution
  - Position this within the overall information “landscape” in which the web application exists.

Example Preliminary Site Map

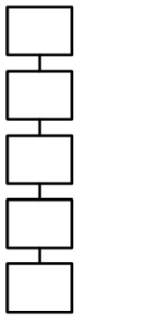


IA Characteristics

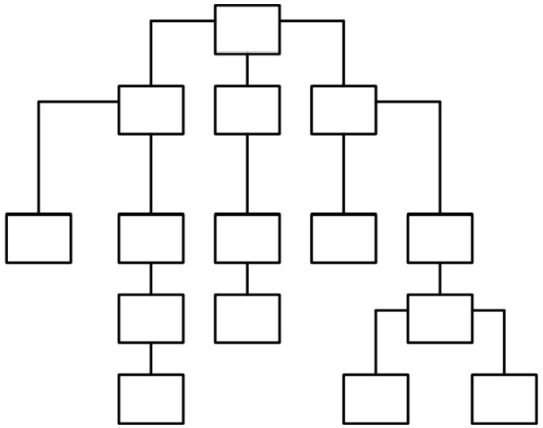
- **Composition with multiple, dynamic data:** The model must support the ability to group different information items into a presentation and the expression of constraints among these items.
- **Higher-level presentation specification:** The model should be able to specify constraints across multiple information items.
- **Temporal relations:** Certain information items may have time-based relationships, which can be important to their presentation (e.g., a link to information about an event might only be relevant up until that event is held).
- **Context for links and link semantics:** The ability to control the presentation depending upon which links are followed.
- **Separation of content and information:** Content is the collection of data sources that are available for use. Information is what is useful to the users of the web application.
- **Separation of information and application:** A web application IA should differentiate between the information that a user would find meaningful, and the structural ways in which this information might be arranged and accessed.
- **Separation of application and presentation:** If we separate the presentation mechanisms from the application, then the portability and genericity of applications (ability to be applied to other applications or problems with minimal change) will be substantially enhanced.

Structuring the Info Space

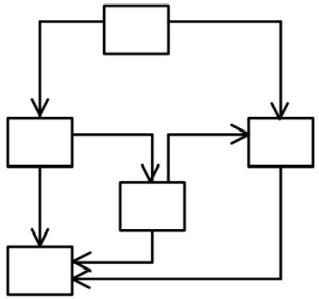
- The information structures that are created during information design can be classified in various ways
  - What application domains do you think are suited to each of these structures.



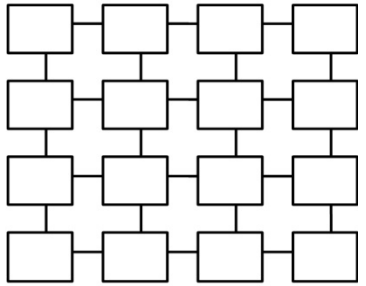
Linear Structure



Hierarchichal Structure



Network Structure



Matrix Structure

What Makes a "Good" Structure

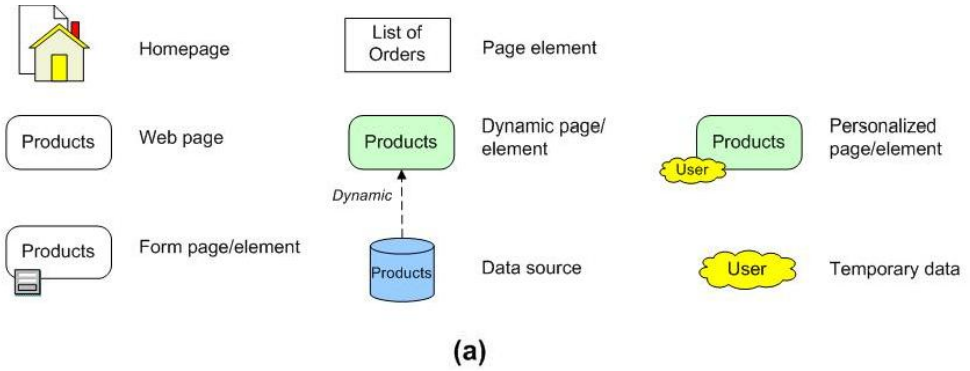
- For hierarchical structures:
  - *Meets the information needs of the users and is easy to navigate!*
  - The breadth and depth of the information structure can have a strong impact on how much effort it takes a user to navigate to information that is needed
  - *The appropriate fan-out of the hierarchical structure should relate to the complexity of the web application options and how distinct the choices ar*e
    - Fan-out is a measure of the width of the navigation structure below a single node.
  - Hierarchies based on exact values and clear categorizations will typically not be ambiguous.

Blueprints: Adding Detail

- Shows how the various content objects map into a specific information structure
  - Captures additional information to a sitemap
- A blueprint might also discuss:
  - Whether content is dynamic or static
  - Whether content is personalized for individual users (and in what ways)
  - What content objects are mapped to which Web pages
  - What navigational paths will address given tasks
- Allows you to visualize how a web application might fit together and, hence, how users might respond to it

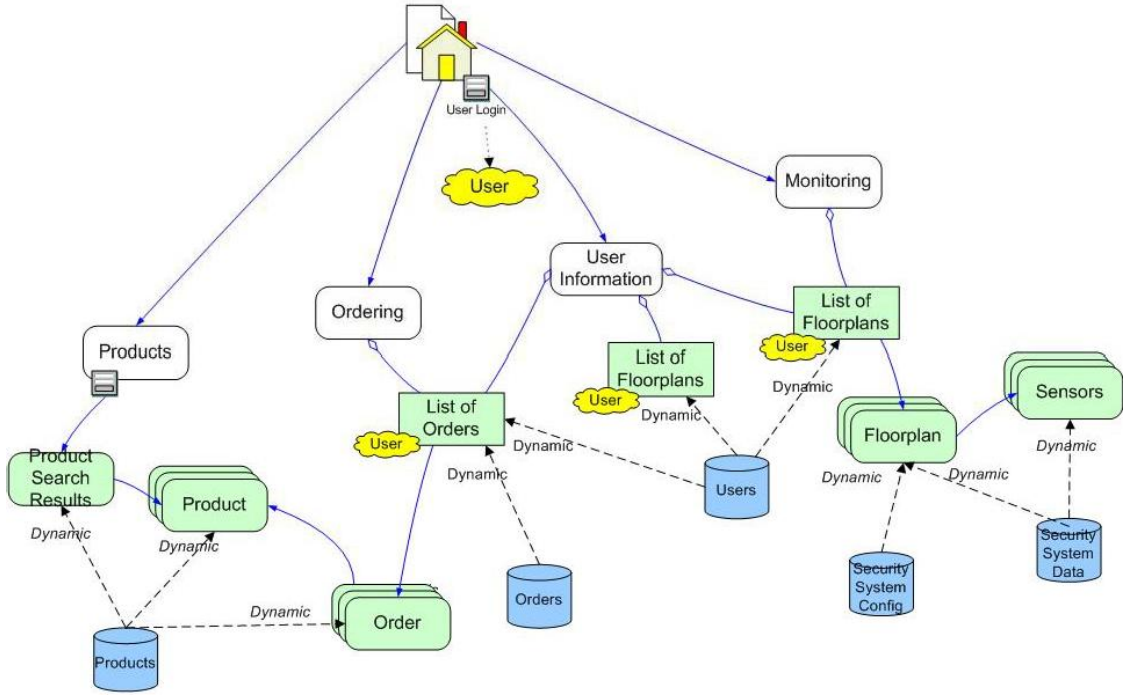
Blueprints

Basic notation



(a)

Example blueprint structure



(b)

Accessing Information

- A number of other factors affect the ability of users to achieve their goals: generally relate to navigational mechanisms and characteristics:
  - Web application mechanisms that allow users to understand what navigation options are available at any given time (e.g. menus)
  - Interface mechanisms that provide users with an indication of where they are and what they are currently seeing (e.g. breadcrumbs)
  - Navigation mechanisms that allow users to travel within the information structure. (e.g. searching)
- Each must be considered as part of the information design

Understanding context

- Have you ever navigated into a complex web application and felt “lost in hyperspace”?
- When this happens, you lose track of where you are within (or beyond) the web application.
- It’s a common problem that can leave the user disoriented and unable to acquire needed information or invoke appropriate functionality.

Defining Context - Guidelines

- **Clear labeling**: Develop a set of local standards that lead to a clear set of labels for all link anchors.
  - Anchors describe the destination of the link and can be crucial for ensuring that users understand where they have landed when following a link
- **Breadcrumbs**: It's always a good idea to know where you've come from as you navigate deep into an information structure
- **Identity**. Each Web page should clearly identify the nature of the site or subsite to which presented information belongs

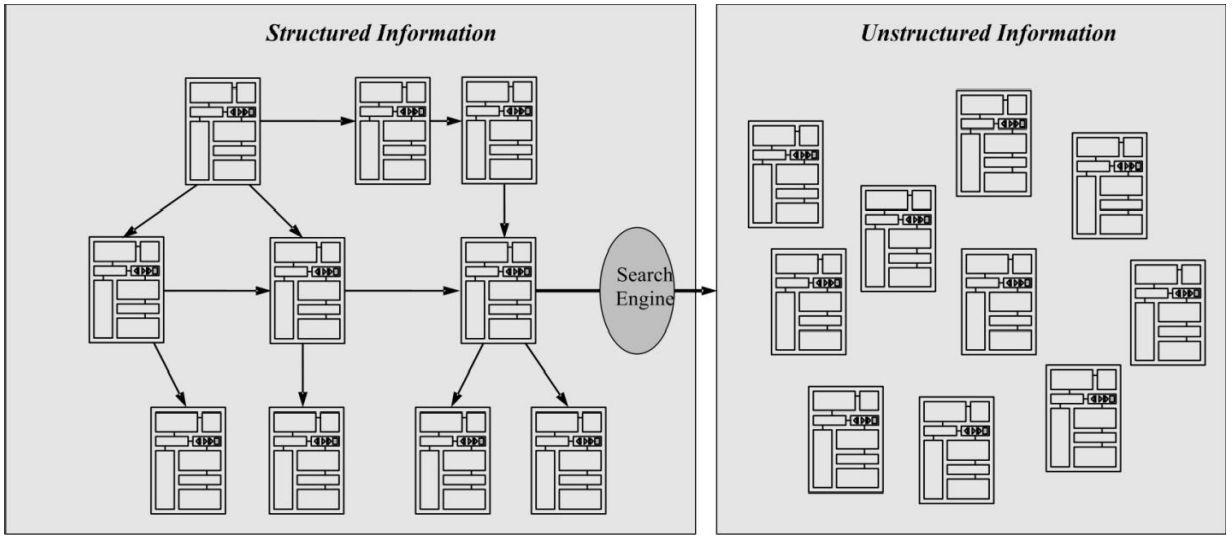
Moving through the Info Structure

- An information architect should:
  - tune navigational support to the specific characteristics of the IA
  - design search mechanisms that lead the user to desired information while filtering out extraneous content.
  - help experienced users achieve their navigational goals more quickly
  - provide inexperienced users with additional navigational support

- Accomplished with:
  - **Global links**: These links are provided on every Web page and point to commonly visited web application locations or functions.
  - **Shortcuts:** These are ways of bypassing the normal navigational route and jumping over intermediate steps straight to a particular location within the information space
  - **Breadcrumbs and trails:** We have already noted that breadcrumbs are useful for helping users to locate themselves.

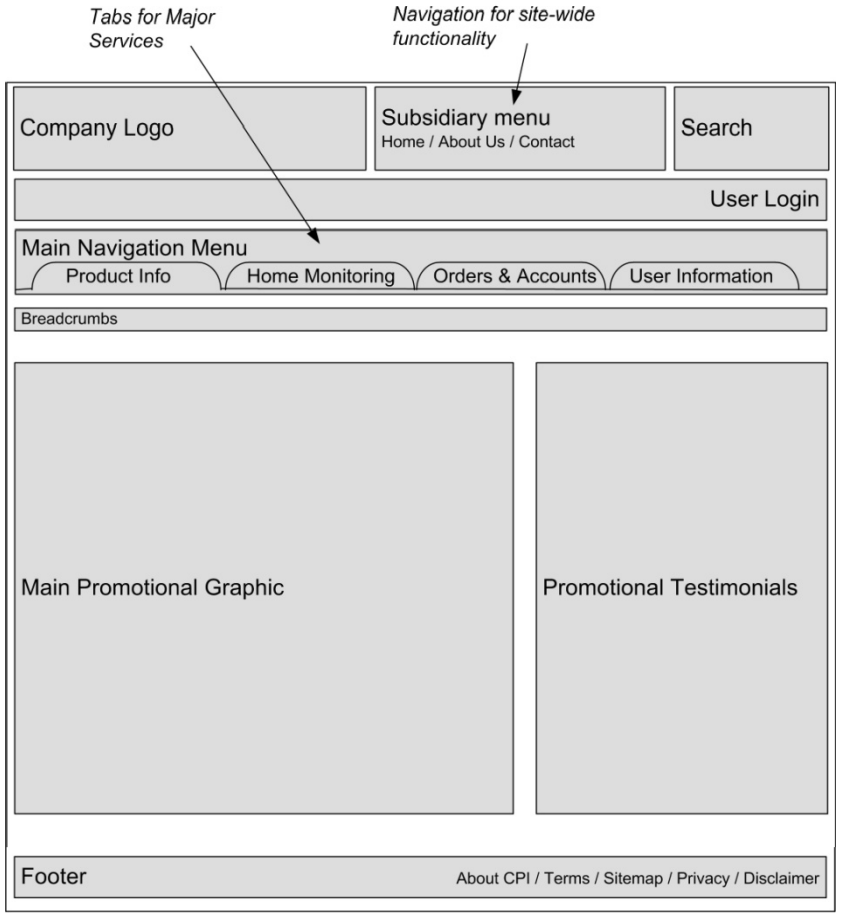
Searching Mechanisms

- Allows a user to bypass the imposed navigational structure and jump directly to specific locations within the web application.
- A search engine can often be used more profitably by constraining its scope.
- Pages on the right of the figure (representing unstructured information) are less amenable to prescribed navigation and therefore become the focus of the search function



Wireframe Models

- Conceptual layout of pages
- Captures core information and navigational elements.
- Supports both information design and interaction design.

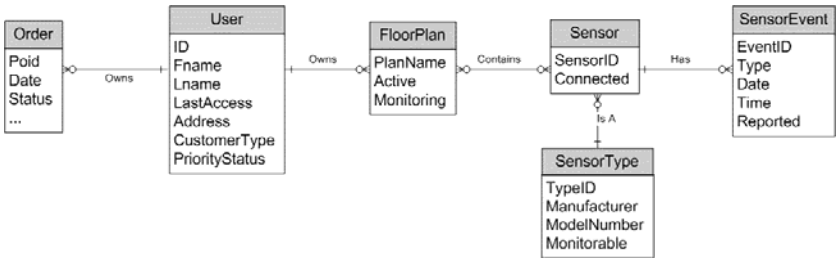


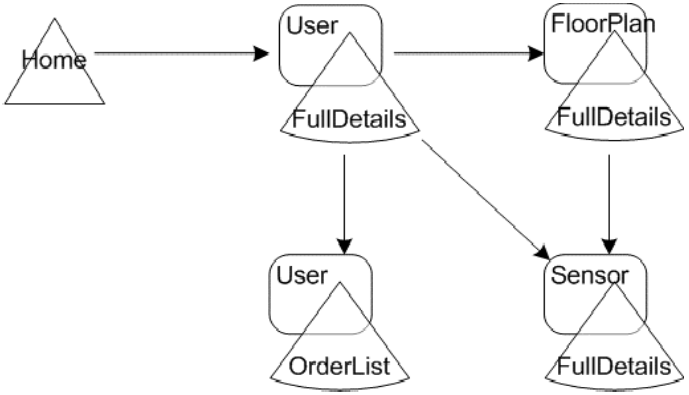
Navigation Design

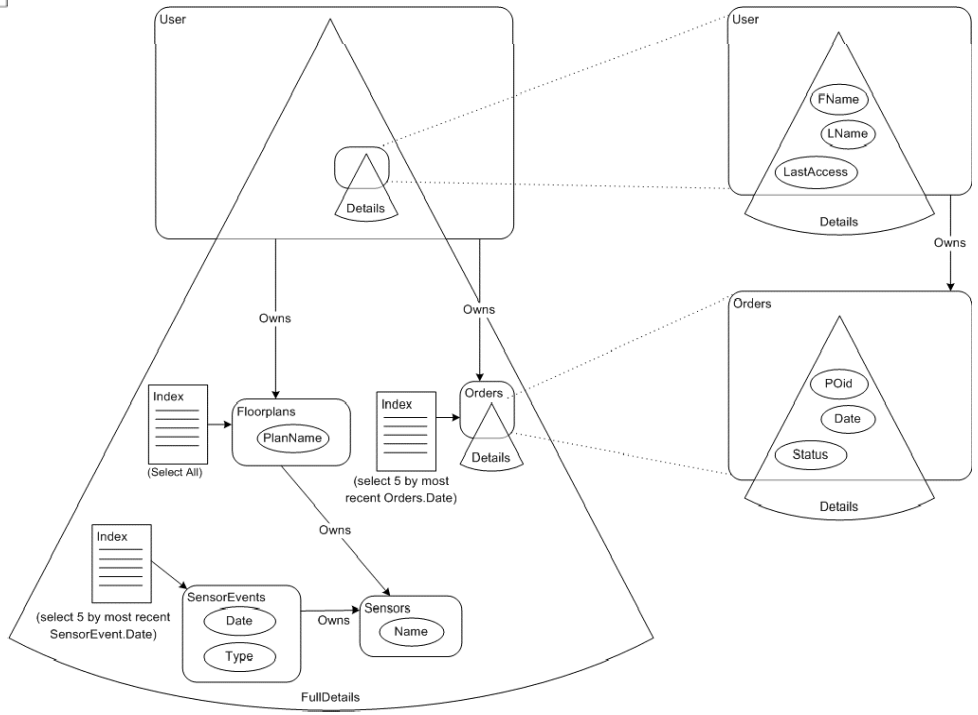
- The *Relationship Management Methodology* (RMM) [Isa95] is an early navigation design approach – useful for illustrating concepts.
  - *ER modeling* defines the information domain of the application by identifying content (data) objects, attributes, relationships, and various type indicators that comprise the web application information space.
  - *Slice design* determines detailed information structure and access mechanisms by grouping content from the domain (captured in the ER model) into collections that can or should be presented together in order to be useful and meaningful
  - *Navigation design* establishes the links between the various slices and creates the information units that have interest for various user categories.

- Ultimately, these information units are aggregated and are transformed into Web pages. The navigation design links these pages by selecting all slices that are the target of a link derived from an ER diagram.

RMM Modeling



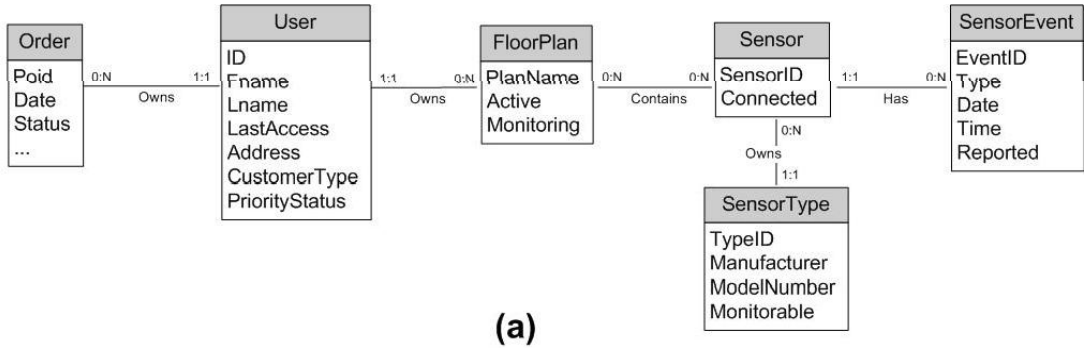




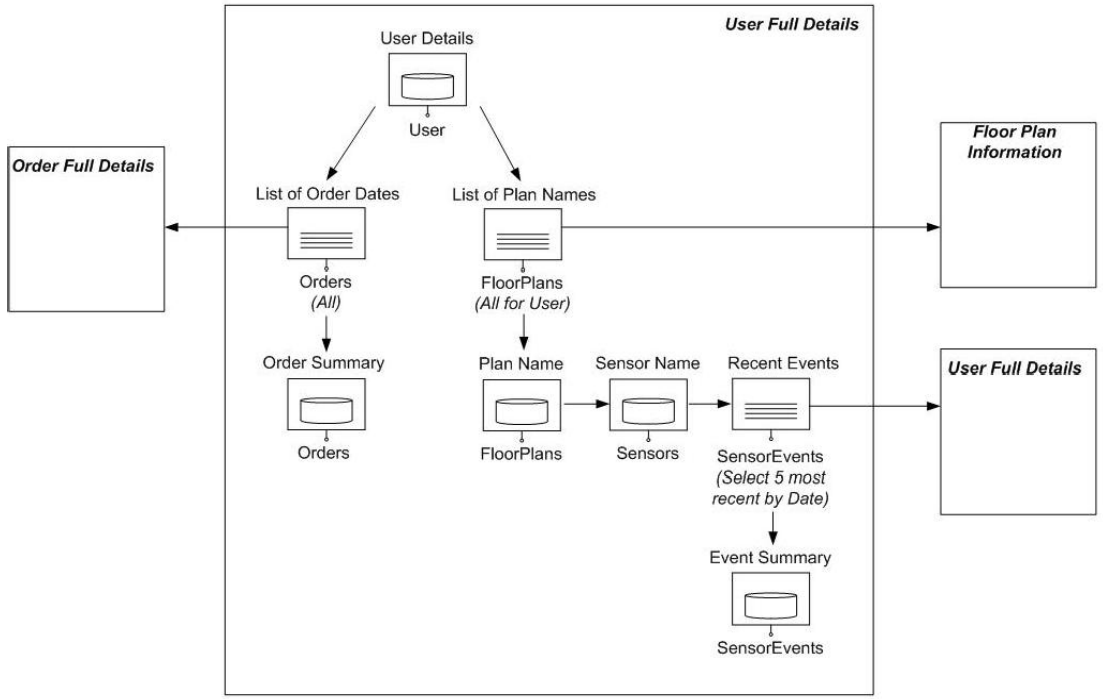
Other Approaches

- A more recently developed, and richer, notation than RMM is the *Web Modeling Language* (WebML)
  - incorporates robust support for aspects such as workflow modeling, presentation and content adaptation, personalization, and design patterns

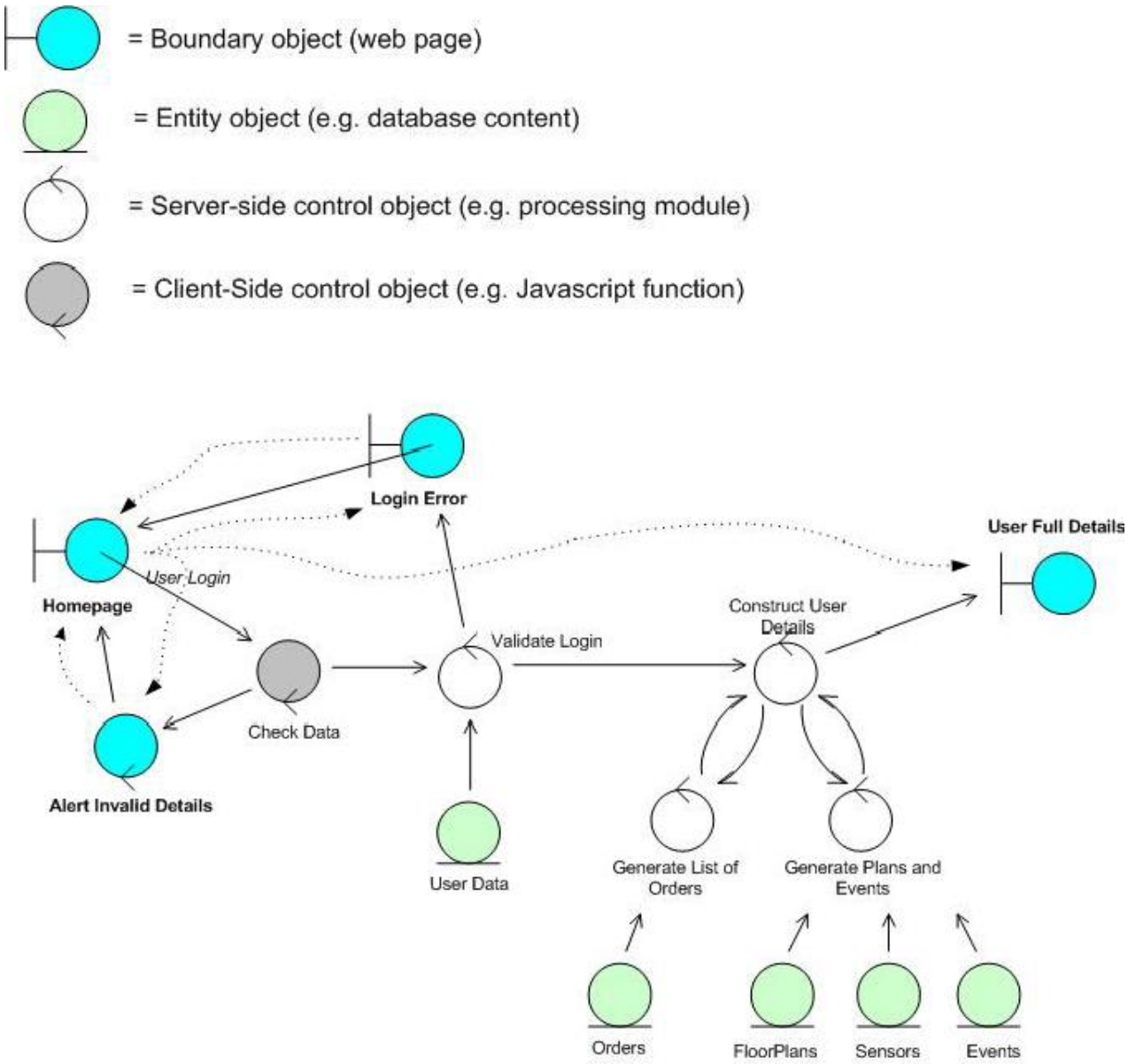
- *Web Application Extension for UML* (WAE) is a design approach that links the informational perspective with functional web application components.
  - Indicates how functional components generate and/or provide information and how the information (through aspects such as link activation or form submission) triggers functional components.
  - Models the connection between client-side content and behavior, and server-side functionality.


Order
Poid
Date
Status
...
0:N
Owns
1:1
User
ID
Fname
Lname
LastAccess
Address
CustomerType
PriorityStatus
1:1
Owns
0:N
FloorPlan
PlanName
Active
Monitoring
0:N
Contains
0:N
Sensor
SensorID
Connected
0:N
Owns
1:1
SensorType
TypeID
Manufacturer
ModelNumber
Monitorable
1:1
Has
0:N
SensorEvent
EventID
Type
Date
Time
Reported

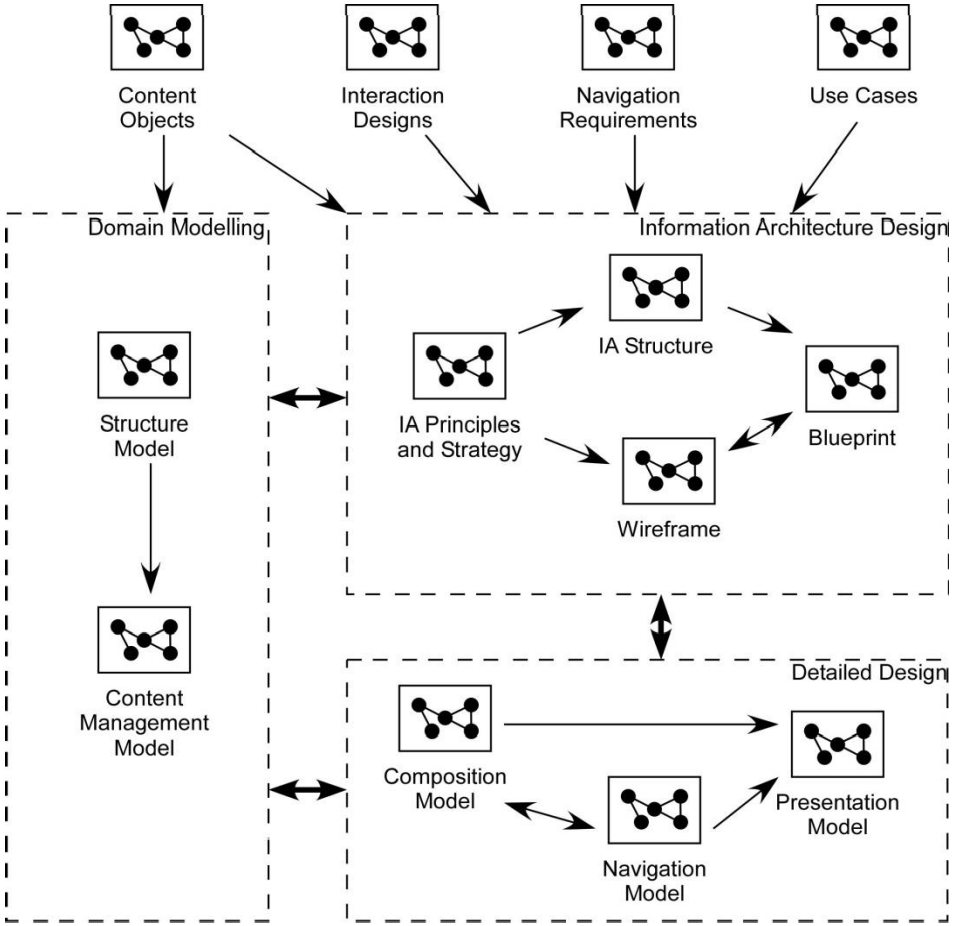
(a)


User Full Details
User Details
User
Order Full Details
List of Order Dates
List of Plan Names
Floor Plan Information
Orders
(All)
FloorPlans
(All for User)
Order Summary
Plan Name
Sensor Name
Recent Events
User Full Details
Orders
FloorPlans
Sensors
SensorEvents
(Select 5 most recent by Date)
Event Summary
SensorEvents

(b)


= Boundary object (web page)
= Entity object (e.g. database content)
= Server-side control object (e.g. processing module)
= Client-Side control object (e.g. Javascript function)
Login Error
User Full Details
User Login
Homepage
Construct User Details
Validate Login
Check Data
Alert Invalid Details
User Data
Generate List of Orders
Generate Plans and Events
Orders
FloorPlans
Sensors
Events

Information Design: Summary



Information Design: Summary

- The formality of the design process should be tuned to the characteristics of the web application.
  - **Application scale:** As size grows, we need to be able to assess the quality of the design *before* construction begins.
  - **Information volatility:** As content becomes more dynamic a clear architecture becomes more important, but detailed models could inappropriately constrain the web application evolution.
  - **Application volatility:** If overall requirements change frequently then focus on those aspects that are known to be stable.
  - **User heterogeneity.** As end-user diversity increases it becomes more difficult to ensure that there is overall consistency in the information structures and information access paths. Consequently, the blueprint increases in importance.
  - **Application criticality:** Web application quality becomes the central focus when a web application is mission critical. Reviews that focus on design work products are a useful tool.
- The decision about the appropriate depth of modeling for a specific web application project should be made early during the design process and not left to an ad hoc decision driven by time pressures.